AN0217
Application Note

AT32 PMSM FOC Incremental Encoder Quick Start Guide

Introduction

This document introduces how to use AT-MOTOR-EVB evaluation board equipped with AT32 motor control library to drive PMSM motor (with an incremental encoder), and details debugging SOP. ArteryTek offers many motor development boards, and this document uses the AT-MOTOR-EVB as an example for illustration.

Applicable products:

Part number	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

List of tables

List of figures

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)
Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Incremental encoder detection

Sensor-based FOC sinusoidal control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Positioning control

Auto tuning functions:

- Motor winding parameters identification
- Current PI control parameters self-tuning

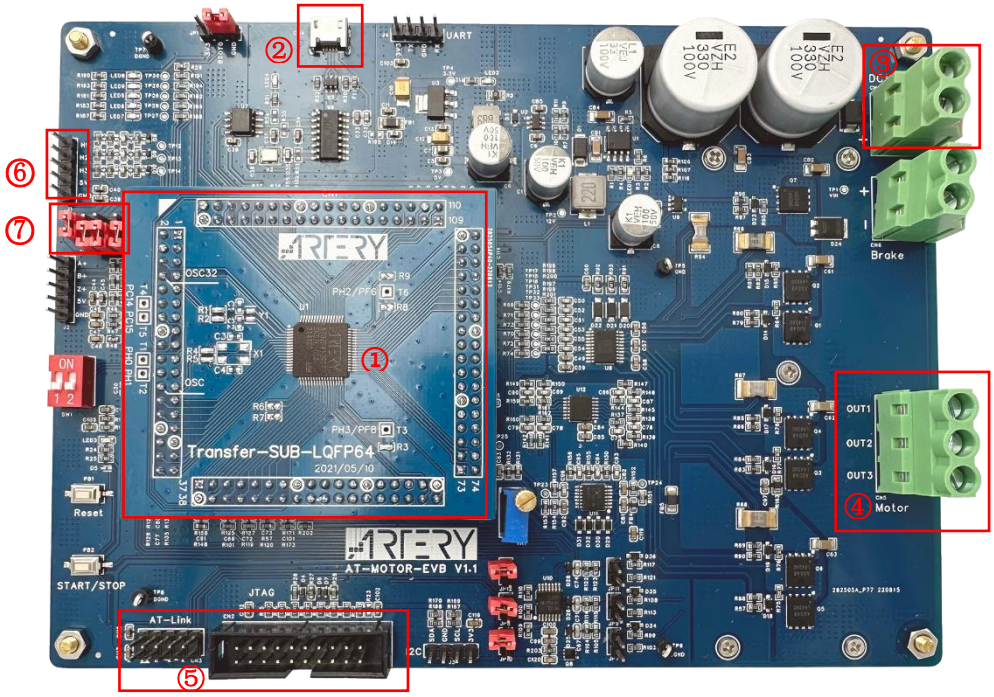
2 Environment requirements

2.1 Hardware requirements

Hardware resources required include a PMSM(BLDC) motor, AT-Link or third-party debugger and a motor control evaluation board (AT-MOTOR-EVB). For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User _Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM(BLDC) motor
- Debugger
- Motor control EVB: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x

Figure 1. AT-MOTOR-EVB V1.x



① AT32 MCU transfer board (part number: AT32F413RCT7)

② USB type B-to- RS-232 connector: connected with PC via USB cable for fine-tuning motor control parameters

③ Power input connector CN1: connected to 24 V

④ Driver output connector CN5: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively

⑤ AT-Link or J-Link connector: for debugging purpose

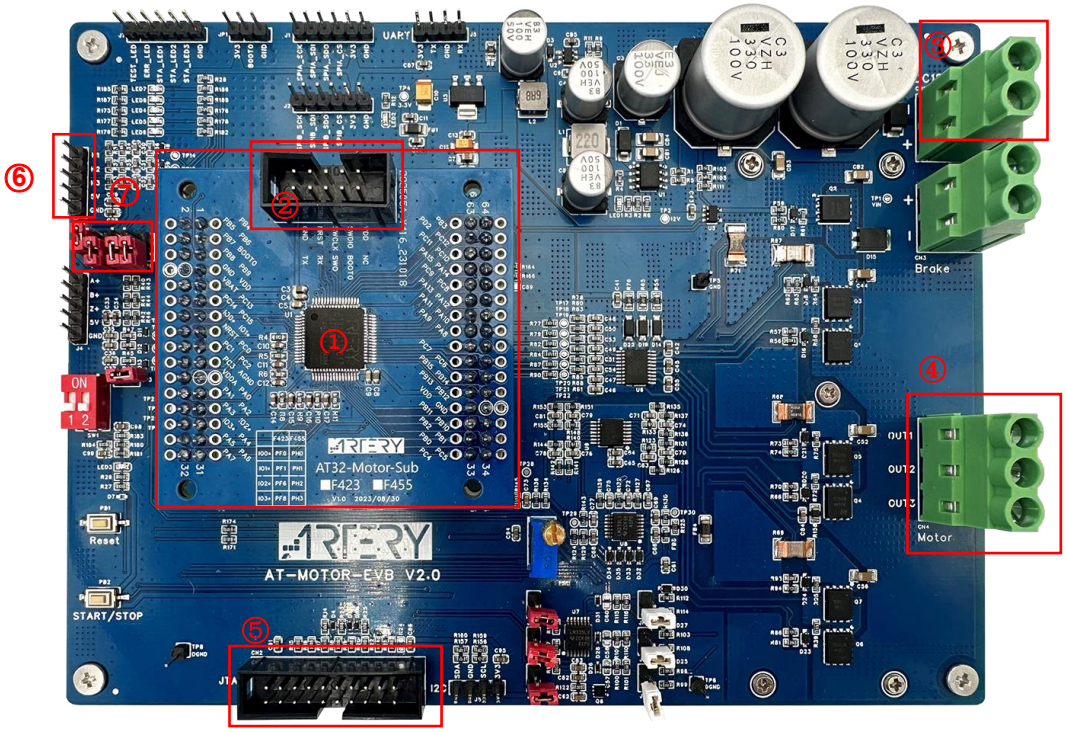
⑥ Incremental encoder connector J2: connected with encoder according to instruction (note [1])

⑦ For AT32F413RCT7 and AT32F415RCT7:
M_METHOD jumper settings: JP2 CLOSED; JP4, JP5 and JP6 OPEN
MT_METHOD jumper settings: JP2, JP4 and JP5 CLOSED; JP6 OPEN

For AT32F403ARCT7, AT32F421C8T7 and AT32F423RCT7:
M_METHOD jumper settings: JP2 CLOSED; JP4, JP5 and JP6 CLOSED.
MT_METHOD jumper settings: not support (note [3])

Note [1]: J1 is not allowed to be connected with hall sensor cable.

Figure 2. AT-MOTOR-EVB V2.x



① AT32 MCU transfer board (part number: AT32F413RCT7)

② AT-Link connector: used for debugging purpose and connecting with UI for motor parameters tuning

③ Power input connector CN1: connected to 24 V

④ Driver output connector CN4: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively

⑤ J-Link connector: for debugging purpose

⑥ Incremental encoder connector J4: connected with encoder according to instruction (note [1])

⑦ Jumper for F422、F421 and L021 AT32-Motor-Sub board

M_METHOD: JP2 CLOSED; JP4, JP5 and JP6 CLOSED

MT_METHOD: NA (Note[3])

Jumper for other AT32-Motor-Sub board

M_METHOD: JP2 CLOSED; JP4, JP5 and JP6 OPEN

MT_METHOD: JP2, JP4 and JP5 CLOSED; JP6 OPEN

Note [1]: J3 is not allowed to be connected with hall sensor cable.

2.2 Software requirements

1) Taking AT32F413 as an example, open AT32F413_MC_Library_Project\motor_evb_2v0 \at32f413\pmsm_foc_incremental_encoder (note [2])

2) Run “ArteryMotorMonitor.exe” (installation not required)

3) In Keil, go to ***Options - Read/Only Memory Areas*** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example, for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 3).
In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4).
In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 1. Flash memory size and ROM configuration

Flash size	**4032K**	**1024K**	**512K**	**448K**	**256K**
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	**128K**	**64K**	**32K**	**16K**
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [2]: This demo does not support using of Keil complier version 5 and O0 optimization level for compiling purpose simultaneously. Thus either Keil compiler version 6 or O1 optimization level is used for this purpose. As AT32 BSP source code does not support V6.15 compiler, it is recommended to use Keil compiler version 5 and O1 optimization level.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

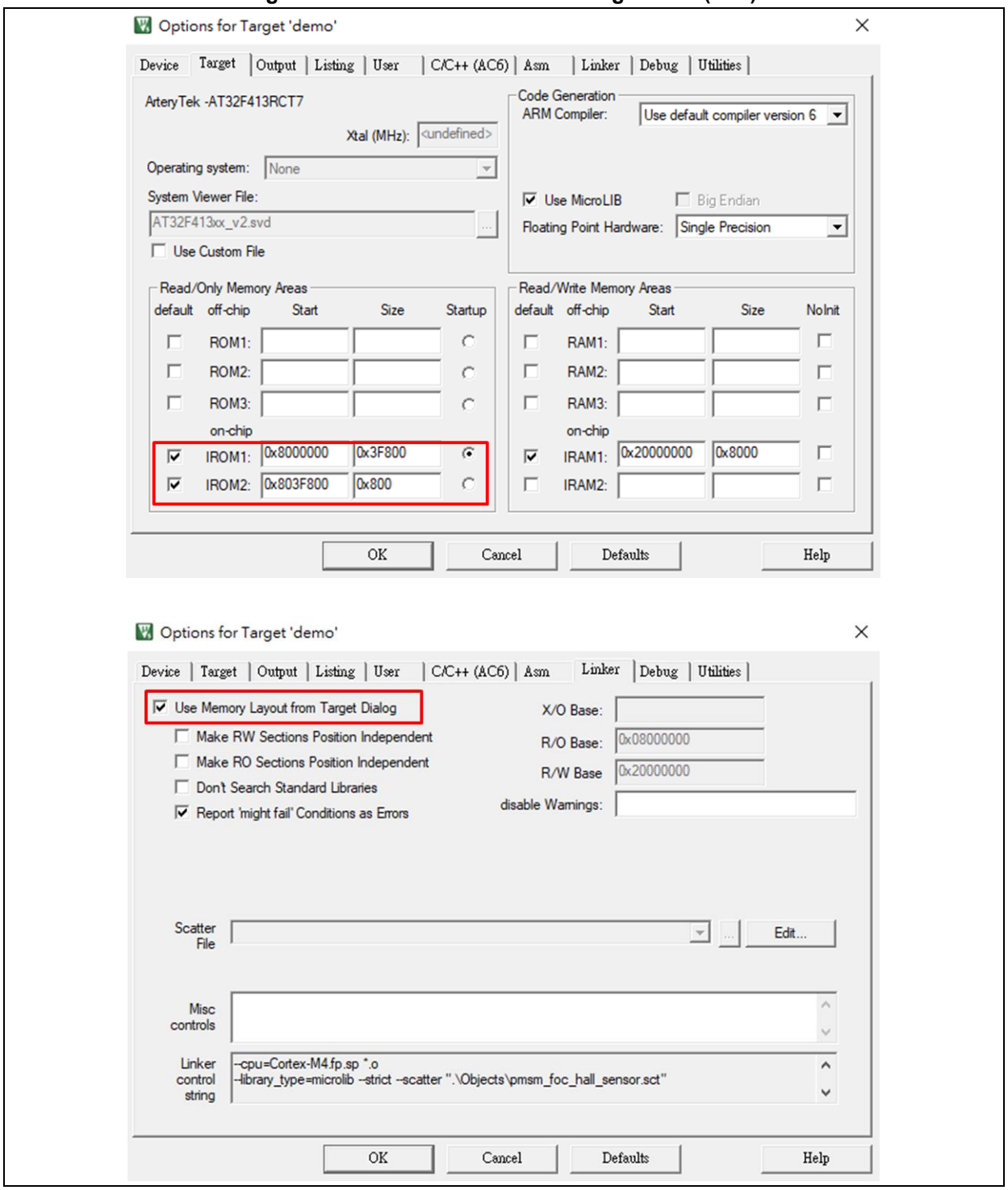
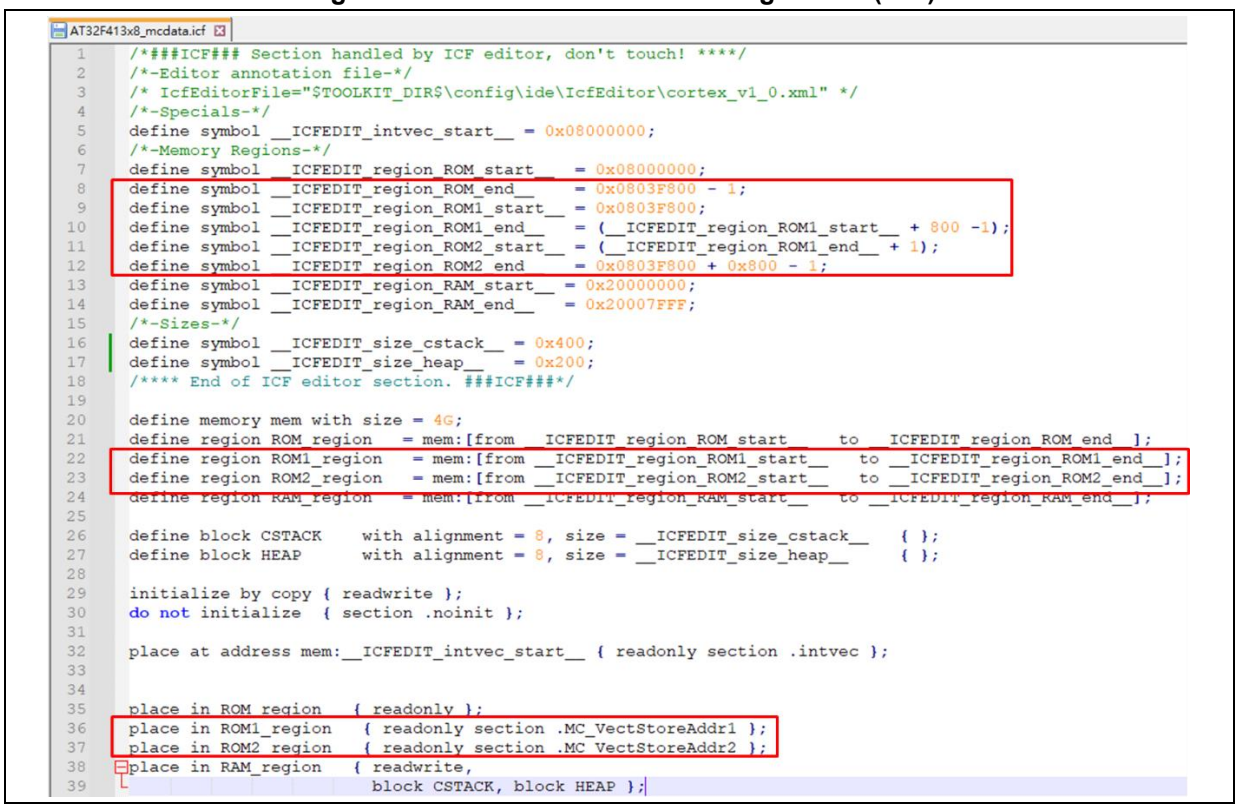


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

AT32F413xC_FLASH.ld

```
_estack = 0x20008000;    /* end of RAM */

/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
MC_DATA (r)     : ORIGIN = 0x0803F800, LENGTH = 0x800
RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
}

/* Define output sections */
SECTIONS
{
  /* The startup code goes first into FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  .mc_data :
  {
    . = ALIGN(4);
    . = ORIGIN(MC_DATA);
    _MC_VectStoreAddr1 = .;
    *(.MC_VectStoreAddr1);
    . = ALIGN(4);
    . = ORIGIN(MC_DATA) + 800;
    _MC_VectStoreAddr2 = .;
    *(.MC_VectStoreAddr2);
    . = ALIGN(4);
  } >MC_DATA

  /* The program code and other data goes into FLASH */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)
```

Figure 5. AT32F413RCT7 ROM configuration (IAR)



3 Quick start guide

Follow the procedures below to fine-tune a PMSM motor with incremental encoder by using FOC control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Select open loop control mode to check whether motor is running normally and to verify the motor's rotation direction and the angular direction of the encoder. (see Section 3.3).

Step 4: Alignment calibration of the incremental encoder is required upon initial motor startup or after control board repowering.(see Section 3.4).

Step 5: Select voltage control mode to check whether the motor spins based on encoder position (see Section 3.5).

Step 6: Current control

1. Motor winding parameter identification (see Section 3.6)
2. Current PI controller parameter auto identification (see Section 3.6)
3. ID tune: fine-tuning Kp and Ki values of current loop (see Section 3.7)
4. IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.8)
5. Current loop control (see Section 3.9)

Step 7: Tune speed control parameters (see Section 3.10).

Step 8: Tune positioning control parameters if necessary (see Section 3.11).

Step 9: Adjust field weakening control parameters as needed (see Section 3.12)

Step 10: (If the motor type is a Surface-mounted Permanent Magnet Synchronous Motor (SPMSM), skip this step.) If the motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM) and requires Maximum Torque per Ampere (MTPA) control and Maximum Torque per Voltage (MTPV) control, it is necessary to perform MTPA/MTPV table creation and look-up table control (as referenced in Section 3.13).

Step 11: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.14).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* header file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configurations, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board (V1 or V2, optional)

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include *mc_hwio_v1.h* file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include *mc_hwio_v1.h* file)

3. Enable low-side MOS inverted output according to the model of gate driver on hardware. For example, U3315 on AT32_LV_MOTOR_CONTROL_EVB supports inverted output; therefore, the low-side MOS inverted output is enabled by default.

Table 4. Macro definition setting (low-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default; comment to disable)

4. This example is a FOC vector control demonstration project, so there is no need to make changes. This option, however, is reserved for use in code judgment.

Table 5. Macro definition setting (vector control mode)

Definition	Description
FOC_CONTROL	Vector control mode

5. Select a current sampling method from below according to user hardware circuit and application requirements.

Table 6. Macro definition setting (current sampling method)

Definition	Description
THREE_SHUNT	3-shunt current sampling
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling

6. If there is a need to use 2-shunt current sampling, select one from below according to user hardware circuit and application requirements; if not, this step is not needed.

Table 7. Macro definition setting (two-shunt current sampling)

Definition	Description
U_V_SHUNT	U & V shunt
V_W_SHUNT	V & W shunt
U_W_SHUNT	U & W shunt

7. Based on whether the user's hardware circuit uses the low-side MOSFET's RDS(on) for current sampling.

Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)

Definition	Description
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$ (disabled by default)

8. Based on whether the user's requirements call for using dual ADCs for current sampling (Applicable to MCUs with dual or more ADCs; not suitable for ONE_SHUNT configurations).

Table 9. Macro definition setting (dual ADCs current sampling)

Definition	Description
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (disabled by default)

9. This example is for incremental encoder project, so the incremental encoder is selected by default, and there is no need to make changes. This option, however, is reserved for use in code judgment.

Table 10. Macro definition setting (incremental encoder)

Definition	Description
INCREM_ENCODER	Incremental encoder

10. Enable or disable reverse encoder counting mode depending on user circuit configuration (see Section 3.3).

Table 11. Macro definition setting (encoder reverse counting)

Definition	Description
REVERSE_ENCODER_COUNT	Encoder reverse counting (disabled by default)

11. Select ABZ if the encoder is with zero index, or AB if the encoder is without index.

Table 12. Macro definition setting (encoder with/without zero index)

Definition	Description
ABZ	AB signal with zero index
AB	AB signal without zero index

12. Select the desired encoder speed measurement method. In most cases, the M_METHOD is selected as this method only needs to capture the number of pulse of the encoder. For more accurate speed measurement, the MT_METHOD is an option, which needs an additional timer to obtain the pulse interval time of the encoder in addition to the pulse number (note [3]).

Table 13. Macro definition setting (speed measurement method)

Definition	Description
M_METHOD	M method for speed measurement
MT_METHOD	M/T method for speed measurement

Note [3]: In the AT-MOTOR-EVB board, it uses JP4 and JP5 CLOSED to connect with another timer to configure MT_METHOD.

13. Enable d/q-axis current signal low-pass filter according to user needs.

Table 14. Macro definition setting (d/q-axis current low-pass filter)

Definition	Description
CURRENT_LP_FILTER	d/q-axis current low-pass filter (it is default disabled for 3-shunt and 2-shunt current sampling; it is default enabled for 1-shunt current sampling)

14. Enable a/b/c phase current signal low-pass filter according to user needs.

Table 15. Macro definition setting (a/b/c phase current low-pass filter)

Definition	Description
AC_CURRENT_LP_FILTER	a/b/c phase current low-pass filter (disabled by default)

15. Based on the user's motor type, select either a Surface-Mounted Permanent Magnet Synchronous Motor (SPMSM) or an Interior Permanent Magnet Synchronous Motor (IPMSM) (choose one to activate)

Table 16. Macro definition setting (motor type)

Definition	Description
SPMSM	Surface-Mounted Permanent Magnet Synchronous Motor
IPMSM	Interior Permanent Magnet Synchronous Motor

16. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), enable the macro definition IPM_MTPA_MTPV_TABLE to generate MTPA/MTPV lookup tables

Table 17. Macro definition setting (MTPA/MTPV lookup generation)

Definition	Description
IPM_MTPA_MTPV_TABLE	MTPA/MTPV lookup generation (disabled by default)

17. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), generate MTPA/MTPV lookup tables and store them in the Id_MTPA_LUT and

Iq_MTPV_LUT matrices within the MTPA_MTPV_table.c file. Enable the macro definition IPM_MTPA_MTPV_CTRL to execute lookup table-based torque control.

Table 18. Macro definition setting (MTPA/MTPV lookup table-based torque control)

Definition	Description
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table-based torque control (eabled by default)

18. If the user's motor type is a Surface Permanent Magnet Synchronous Motor (SPMSM), the flux-weakening function can be optionally enabled during speed control based on user requirements.

Table 19. Macro definition setting (filed weakening control)

Definition	Description
FIELD_WEAKENING	Filed weakening control (disabled by default)

19. Enable speed limit feature during torque (current) control according to user needs.

Table 20. Macro definition setting (speed limit during torque control)

Definition	Description
TORQUE_CTRL_WITH_SPEED_LIMIT	Speed limit during torque (current) control (disabled by default)

20. Enable current decouple feature during torque (current) control according to user needs.

Table 21. Macro definition setting (current decouple control)

Definition	Description
CURRENT_DECOUPLE_CTRL	Current decouple control (N/A for AT32L021 series) (disabled by default)

21. Enable winding parameter identification. Once enabled, it is used to detect RS and LS of motor windings, and calculate current loop PI control parameters (see Section 3.6).

Table 22. Macro definition setting (motor winding parameter identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameter identification (enabled by default; comment to disable)

22. Enable or disable the Artery motor UI tool according to user needs.

Table 23. Macro definition setting (Artery motor UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor UI tool (enabled by default; comment to disable)

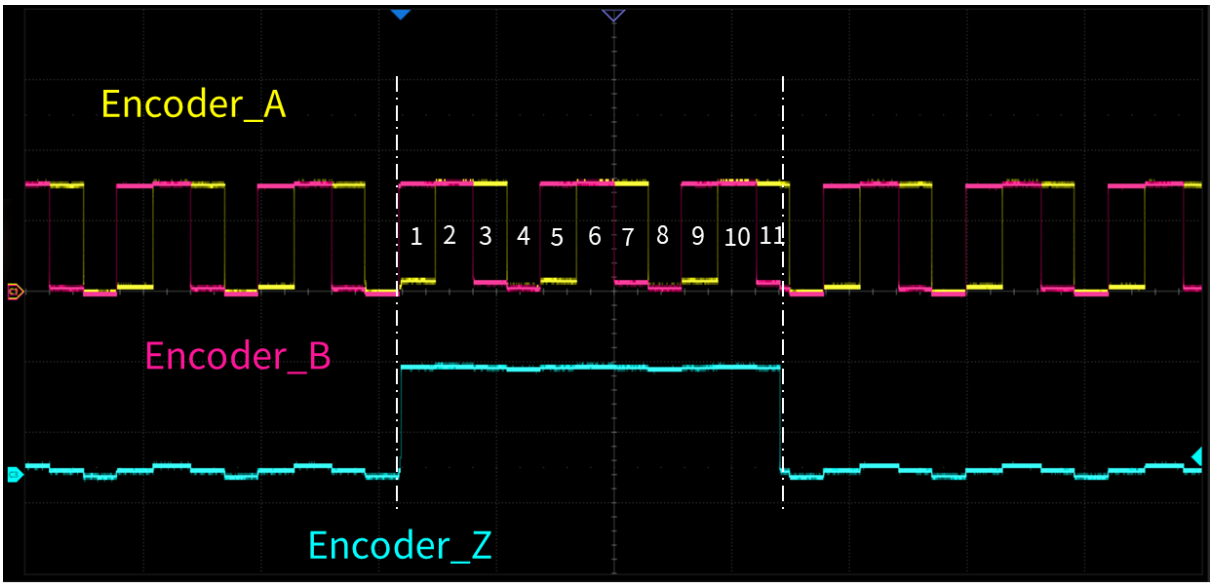
- Table 24 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 24. Motor parameter definition

Definition	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value (unit: V/rpm) (fill in this value when current decouple control is enabled)
LD_LQ_RATIO	Ld to Lq ratio
NOMINAL_CURRENT	Nominal current (unit: ampere)
ENCODER_PPR	Encoder pulse per revolution (unit: pulse per revolution)
ENC_IDX_COUNT	Encoder index signal width (unit: count) (note [4])
ENC_STALL_TIME	Encoder stall time (unit: ms)

Note [4]: This applies to ABZ signal with zero index mode of the incremental encoder. Figure 6 illustrates the relationship between ABZ signals of the JK42BLS01-X056ED encoder. The width between the rising edge and falling edge of the Z signal is 11 count aligned with AB signals, so the ENC_IDX_COUNT is set to 11 (typically, the incremental encoder is one or two count).

Figure 6. Relationship diagram between ABZ signals



3.2 Connection settings

After the hardware and software are well prepared, set up connection between UI and the control board as follows (see AN0063 for details).

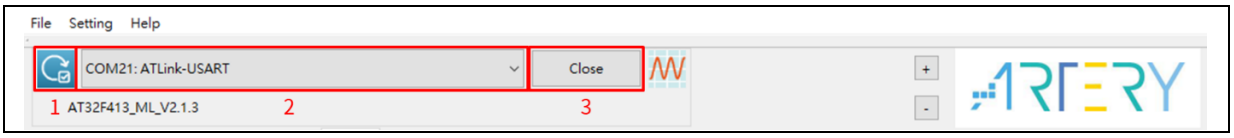
STEP-1
Connect the motor, AT-Link/J-Link and board power supply to the motor development board, and connect USART interface to PC via an USB cable.

STEP-2
Use MDK to compile demo project code, and use J-Link or AT-Link to download to the on-board chip.

STEP-3
Run ArteryMotorMonitor.exe
Click File -> Open Project and select ArteryMotorMonitor.atmcx-> Open.

STEP-4
Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 7.

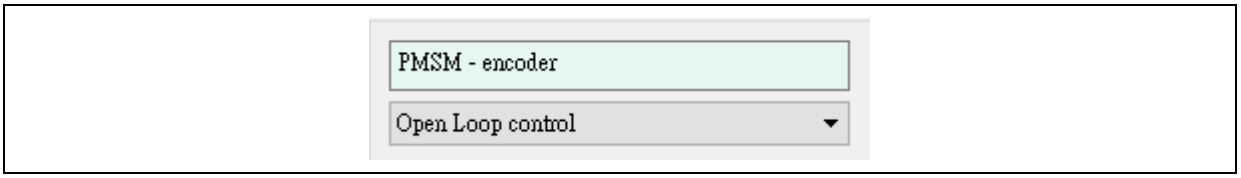
Figure 7. Connection settings



3.3 Open loop control

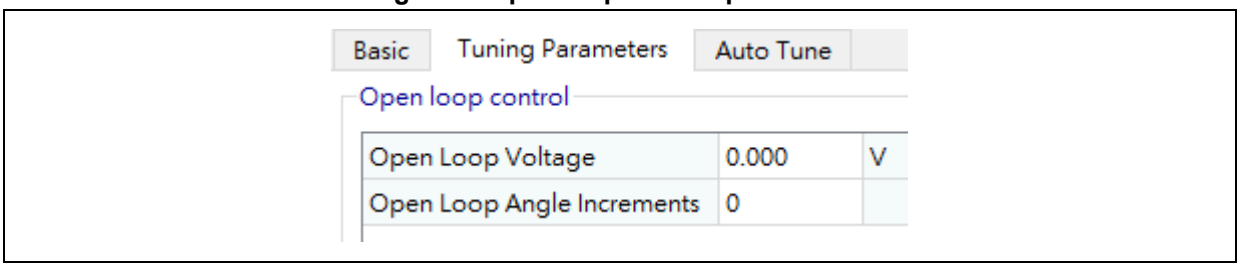
Using open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction, as well as the encoder mode's rotational accuracy. The open-loop voltage and angular increment are adjusted incrementally from zero, based on the motor's speed and phase current.
STEP-1: Select “Open loop control” mode.

Figure 8. Select open loop control mode



STEP-2: Go to “Tuning Parameters” to set “Open Loop Voltage” and “Open Loop Angle Increments”. Click on “Start Motor” and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

Figure 9. Open loop control parameters

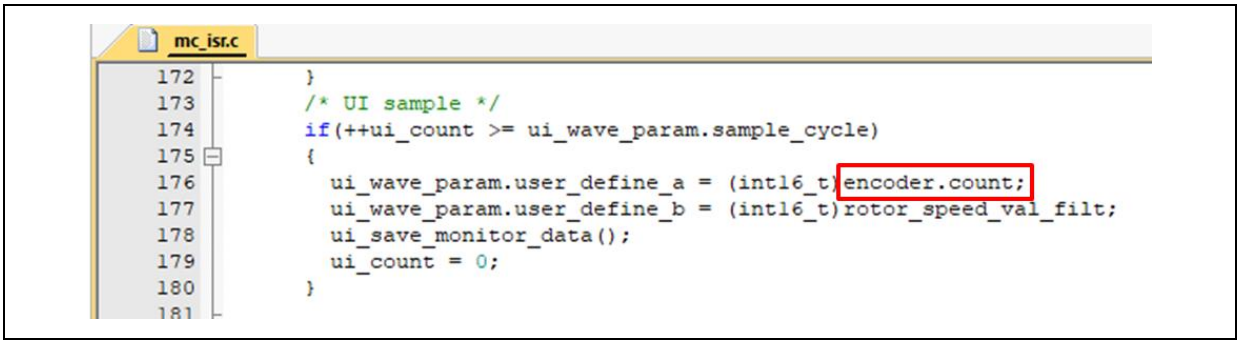


Tips:
Initially, set “Open Loop Voltage” value to 0.5~2V and set “Open Loop Angle Increments” value to 1~10, and monitor whether the motor starts to run. If not, check whether the 3-phase cables are correctly connected.

STEP-3: After the motor starts to run properly, check if it is running in the right direction. If it is reversed, the user needs to change the two phases in the three phases.

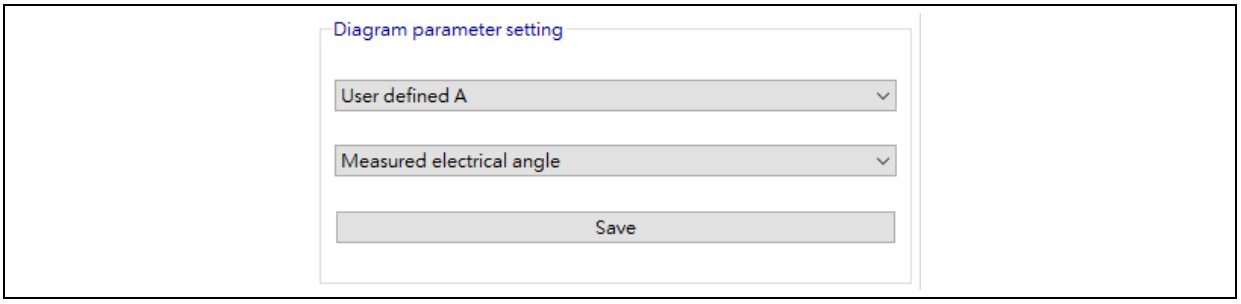
STEP-4: Confirm whether the encoder direction is correct.
STEP-4-1. Modify “ui_wave_param.user_define_a = (int16_t)encoder.count;” in the *mc_isr.c* file, and then recompile and re-program the code

Figure 10. Modify “ui_wave_param.user_define_a” in mc_isr.c file



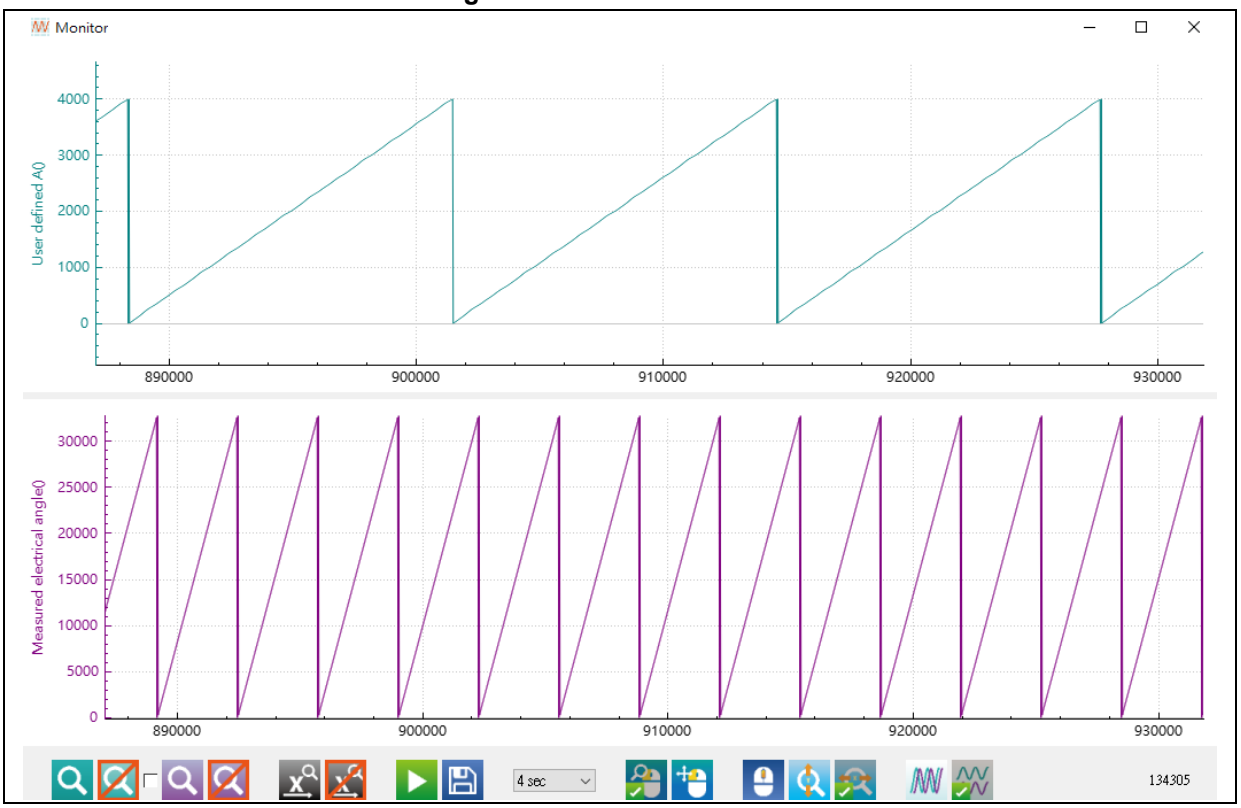
STEP-4-2: Set “User defined A” and “Measured electrical angle”, and click on “Save”.

Figure 11. Diagram parameter setting (confirm encoder direction)



STEP-4-3: After the motor starts to run in open loop control mode, check whether the “User defined A” is the shape of right triangle as shown in Figure 12. If not, it means that the encoder direction is reversed, and users need to change its direction by defining the “REVERSE_ENCODER_COUNT” (Table 11) in the *motor_control_drive_param.h* file.

Figure 12. Encoder direction

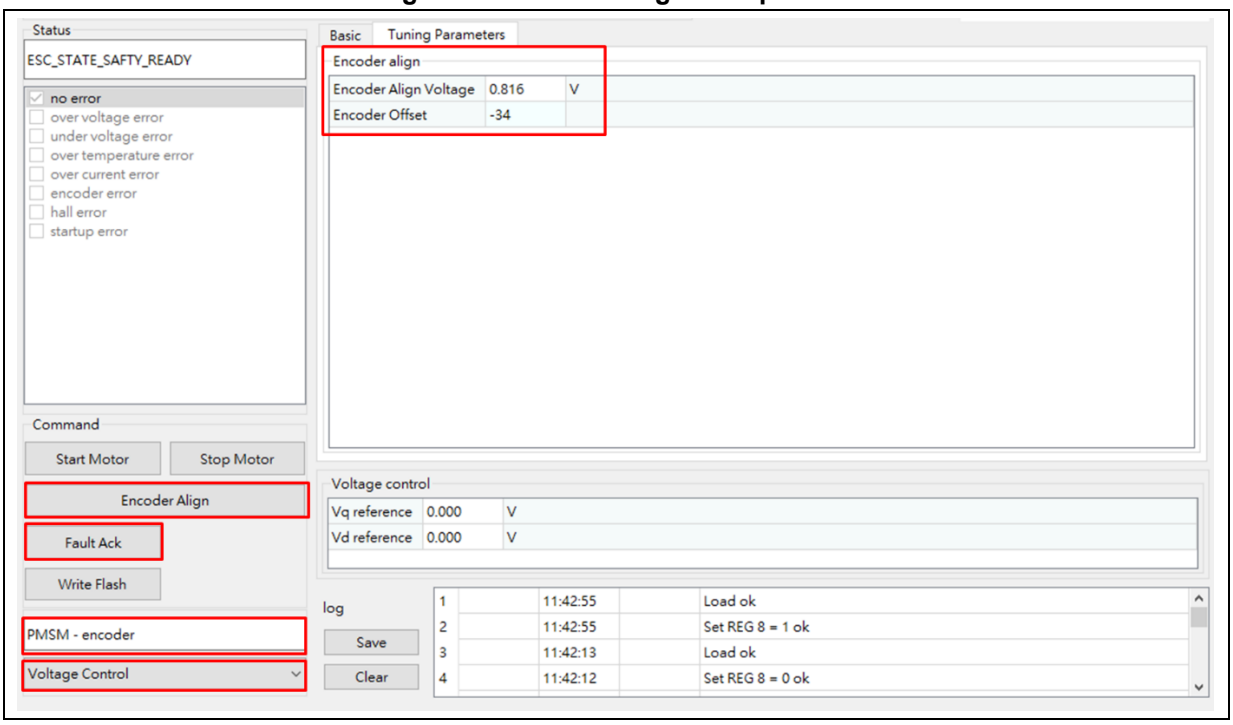


3.4 Incremental encoder alignment

After the first startup of motor or the evaluation board is powered on, it is necessary to align the incremental encoder. The first step is to check whether the firmware control mode is PMSM-encoder, as shown in Figure 13. Select “Voltage Control” mode and set the appropriate encoder align voltage, and click on “Encoder Align” to get started Alignment session. If AB (AB signals without zero index) is selected, the motor is fixed on the D-axis for performing encoder alignment (no offset value). If ABZ (AB signals with zero index) is selected, the motor keeps slowly running until it detects the zero index before stop, that is, the end of the encoder alignment (see note [5]). Users can check the encoder offset to confirm whether it is getting closer to the aligned value. If there is big difference, the user can fine-tune the voltage and re-start aligning.

Note [5]: If encoder error occurs, it is necessary to check whether the motor’s encoder cable is being connected with the encoder interface of evaluation board. In case of error, it is possible to remove it by pressing “Fault Ack” button.

Figure 13. Encoder alignment process

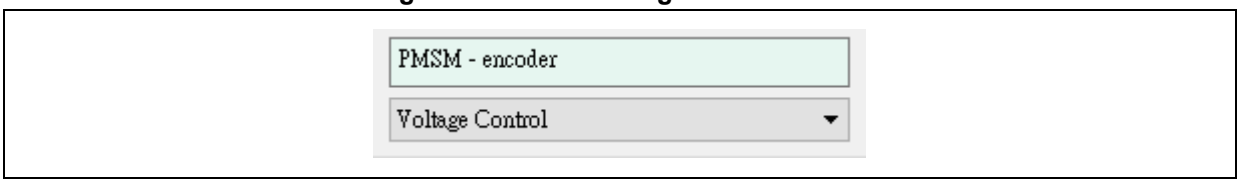


3.5 Voltage control

After the encoder alignment is complete, it is possible to perform voltage control based on encoder position.

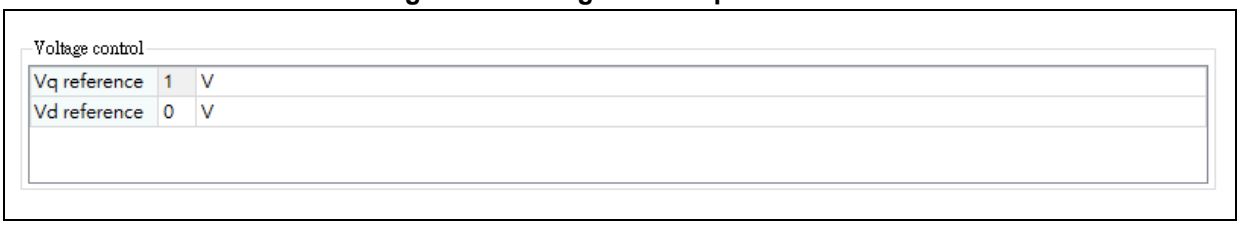
STEP-1: Select “Voltage Control” through the drop-down menu.

Figure 14. Select voltage control mode



STEP-2: Use Q-axis voltage control to drive the motor, and view whether the Ia and Ib values match the actual currents, whether the current offset value is closer to 0 and whether the Ia phase leads Ib phase by 120 degrees. (See Figure 35).

Figure 15. Voltage control parameters

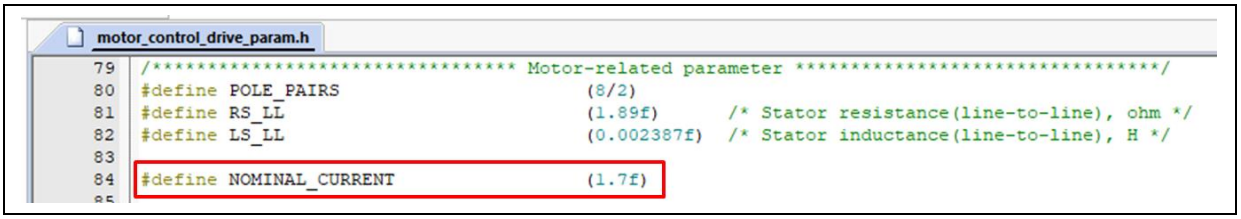


3.6 Auto identification and tuning of parameters

Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters.
Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

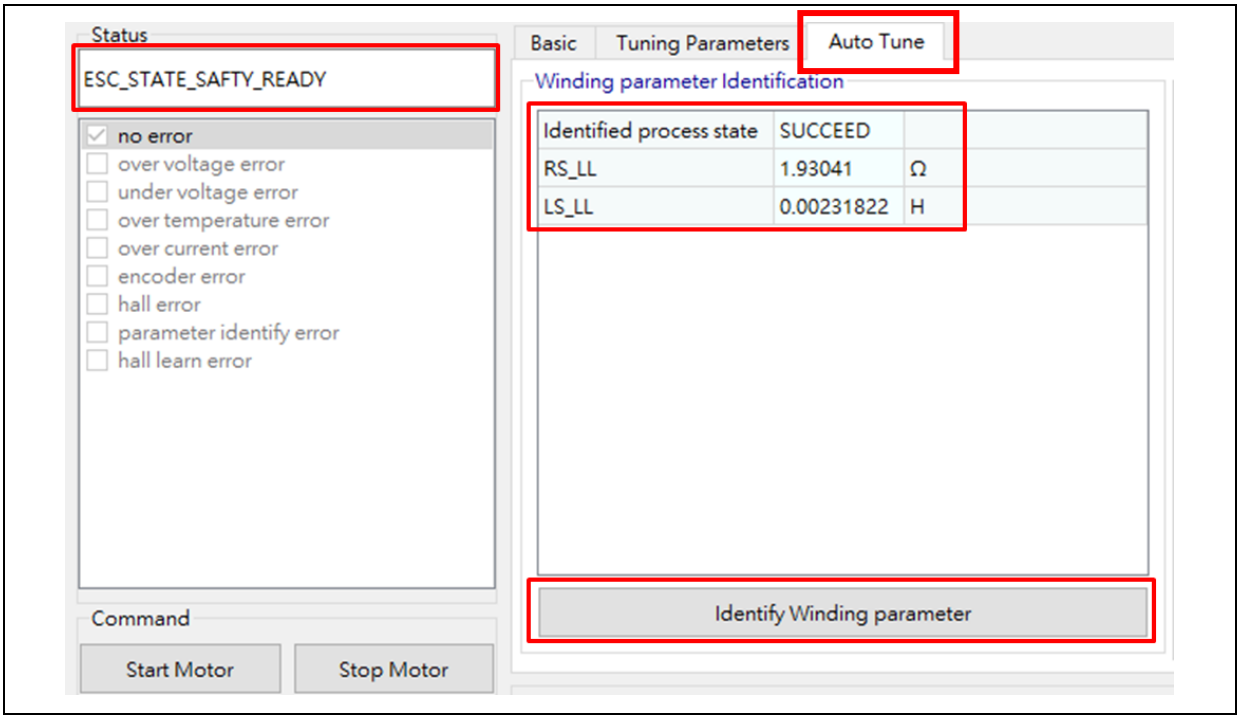
Auto identification of motor winding parameters:
STEP-1: Fill the macro definition NOMINAL_CURRENT into *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 16.

Figure 16. Macro definition of motor nominal current



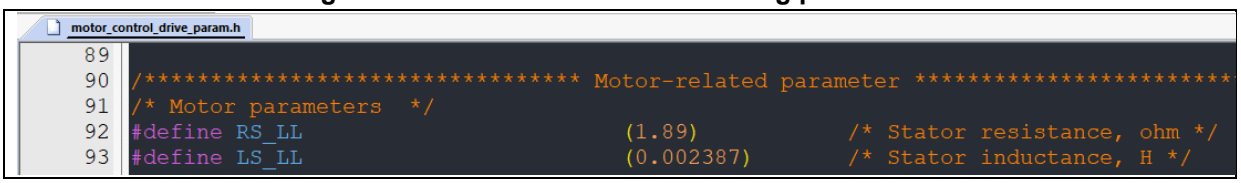
STEP-2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” in “safety ready” status to perform winding parameter identification, as shown in Figure 17.

Figure 17. Auto identification of winding parameters



STEP-3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 18.

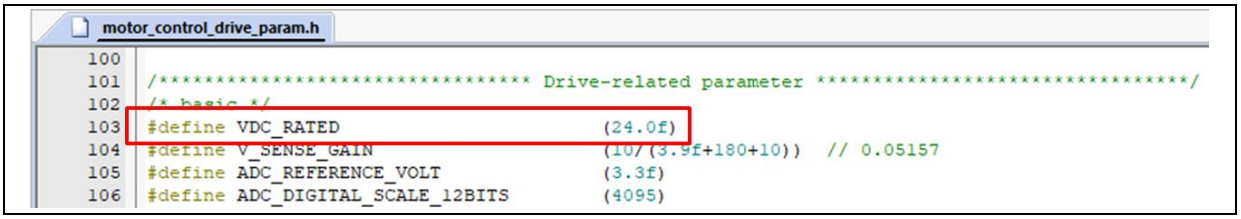
Figure 18. Macro definitions of winding parameters



Current PI controller parameter auto tuning:

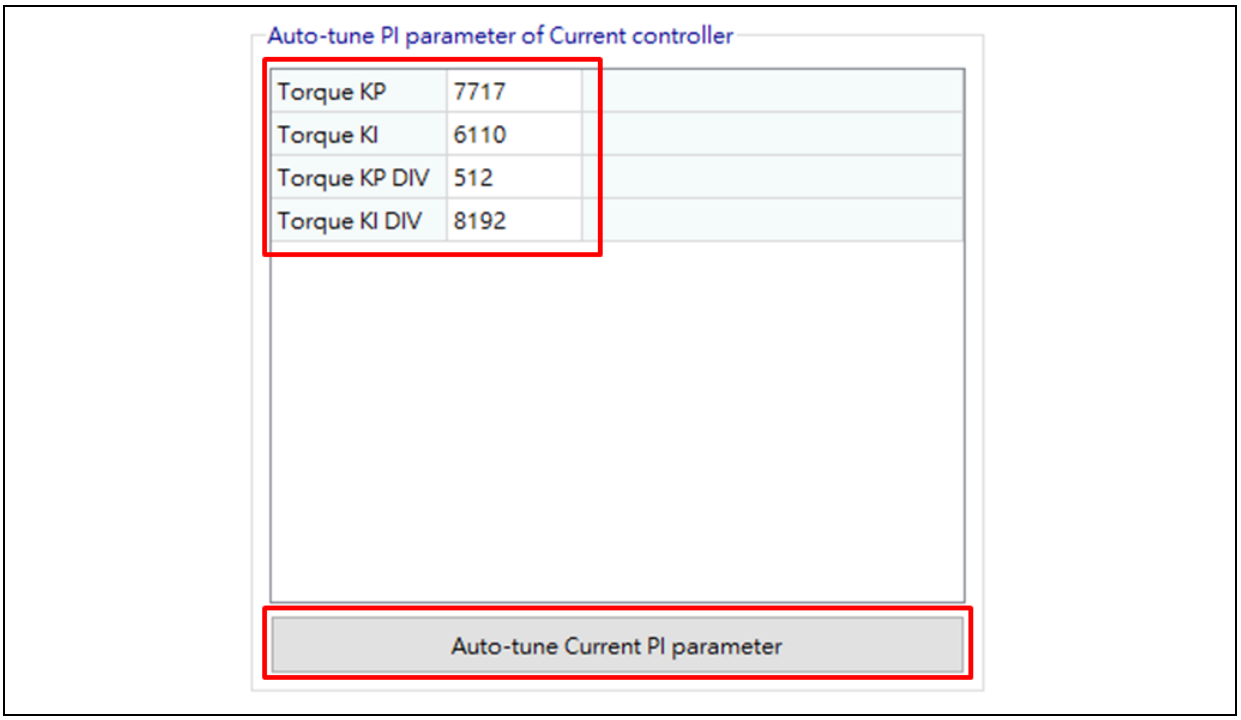
STEP-1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 19.

Figure 19. Motor rated voltage



STEP-2: Go to Auto Tune window, and click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, current Q axis and D axis PI controller parameters are automatically updated, as shown in Figure 20.

Figure 20. Current PI controller parameter auto tuning



STEP-3: Fill the parameters shown in Figure 20 into the corresponding macro definition (motor_control_drive_param.h), then recompile and re-program code, as shown in Figure 21.

Figure 21. Macro definition of current PI controller parameters

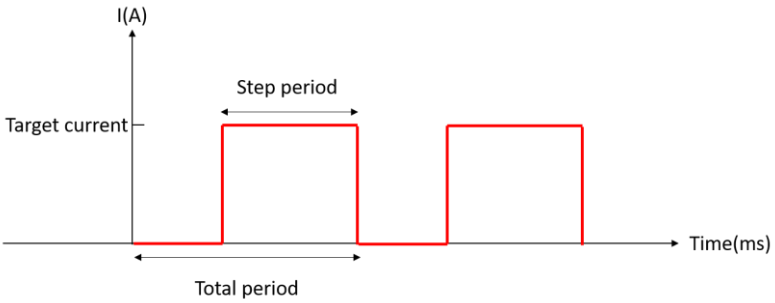
motor_control_drive_param.h

```
244  /* Id/Iq pid parameter */
245  #define PID_ID_KP_DEFUALT              (7717)
246  #define PID_ID_KI_DEFUALT              (6110)
247  #define PID_ID_KP_GAIN_DIV             (512)
248  #define PID_ID_KP_GAIN_DIV_LOG         (LOG2(PID_ID_KP_GAIN_DIV))
249  #define PID_ID_KI_GAIN_DIV             (8192)
250  #define PID_ID_KI_GAIN_DIV_LOG         (LOG2(PID_ID_KI_GAIN_DIV))
251
252  #define PID_IQ_KP_DEFUALT              (7717)
253  #define PID_IQ_KI_DEFUALT              (6110)
254  #define PID_IQ_KP_GAIN_DIV             (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG         (LOG2(PID_IQ_KP_GAIN_DIV))
256  #define PID_IQ_KI_GAIN_DIV             (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG         (LOG2(PID_IQ_KI_GAIN_DIV))
```

3.7 ID tune

A step current is generated in ID Tune mode, as shown in Figure 22. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of D axis current.

Figure 22. Schematic diagram of step current

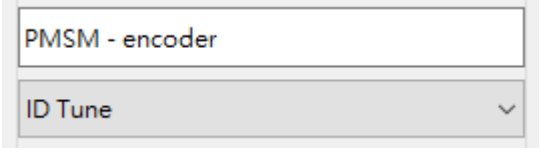


Note: It is recommended to tune D-axis current and then Q-axis current, and to set KP and KI to 0 for D-axis and Q-axis current to avoid unmatched default values affecting current parameter adjustment.
Note: It is recommended to set D-axis current command to 5-10% of the rated current and then slowly increase the D-axis current to the target value to align the rotor with D-axis, so that to protect IPM or MOSFET components from being burnt out by current surges.

Follow the steps bellow:

STEP-1: Select “ID tune” through the drop-down menu.

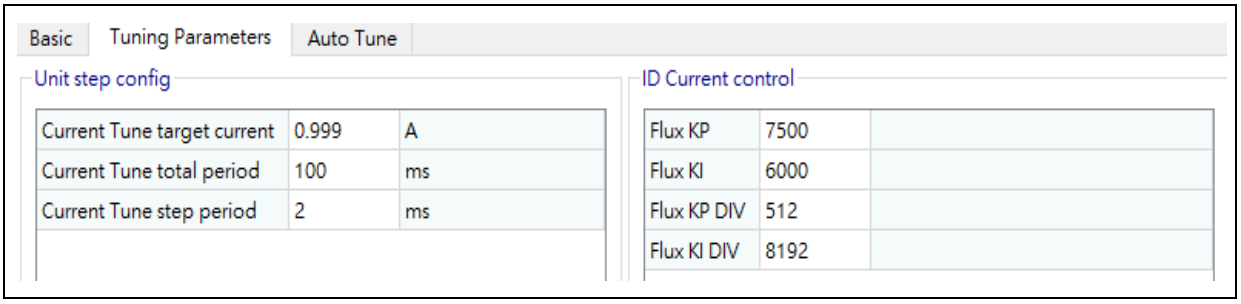
Figure 23. Select ID tune



STEP-2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial

adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 24.

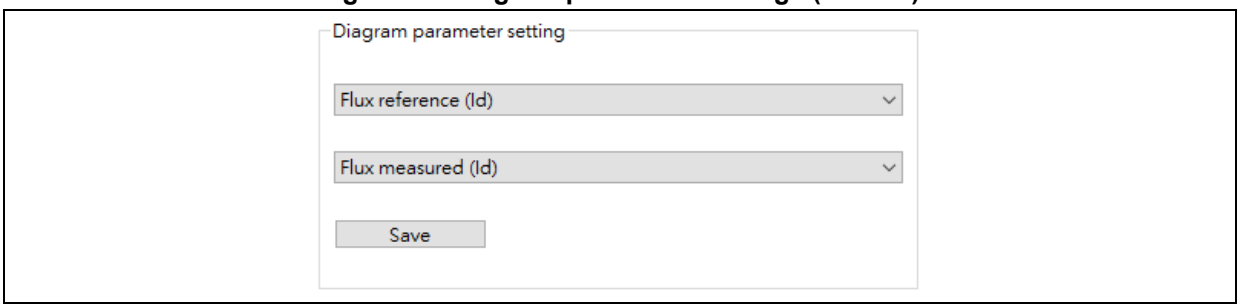
Figure 24. D-axis current PID and step current parameters



STEP-3: Click on “Start Motor”.

STEP-4: Set “Flux reference(Id)” and “Flux measured(Id)” in “Diagram parameter setting”, and then click on “Save”.

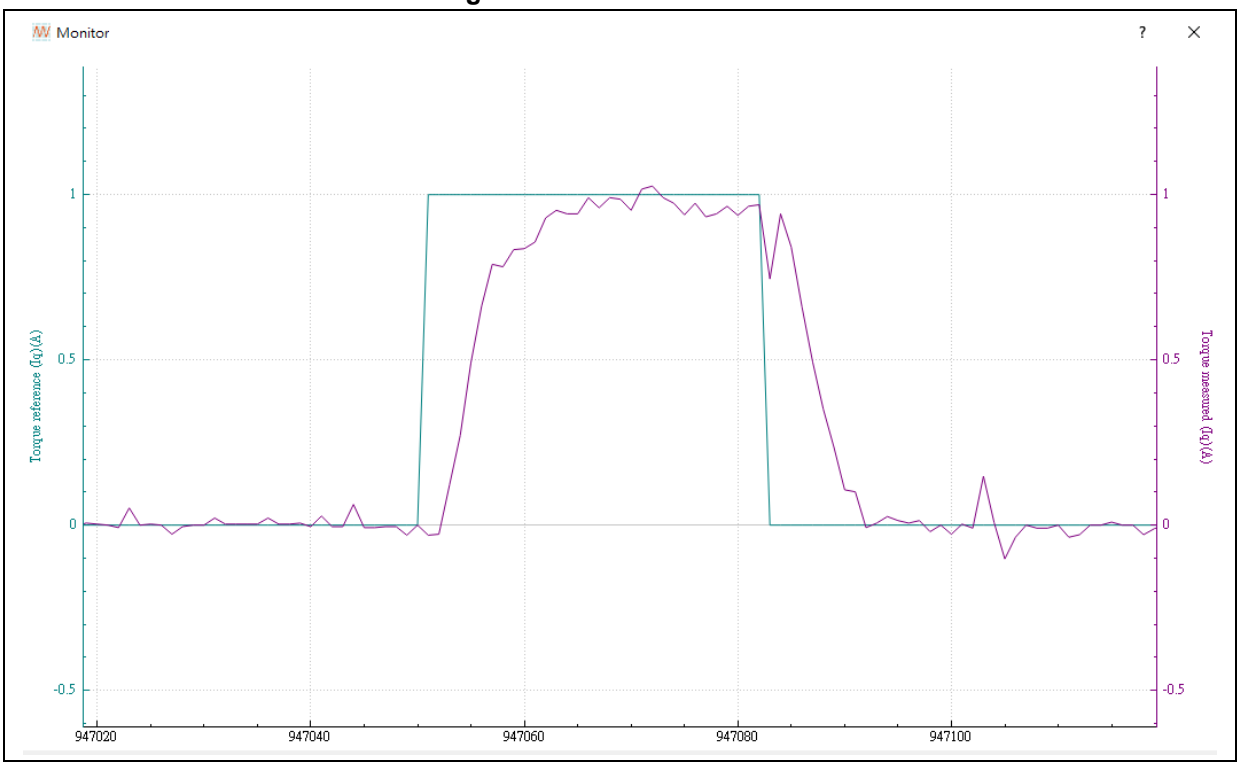
Figure 25. Diagram parameter settings (ID tune)



STEP-5: Click on  icon to generate waveforms.

STEP-6: Check whether the current response is as expected, as shown in Figure 26. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 26. ID tune waveform



STEP-7: After ID tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 27.

Figure 27. D-axis current PI control parameter macro definition

motor_control_drive_param.h

```
245 #define PID_ID_KP_DEFUALT              (7500)
246 #define PID_ID_KI_DEFUALT              (6000)
247 #define PID_ID_KP_GAIN_DIV             (512)
248 #define PID_ID_KP_GAIN_DIV_LOG         (LOG2(PID_ID_KP_GAIN_DIV))
249 #define PID_ID_KI_GAIN_DIV             (8192)
250 #define PID_ID_KI_GAIN_DIV_LOG         (LOG2(PID_ID_KI_GAIN_DIV))
```

3.8 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 22. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

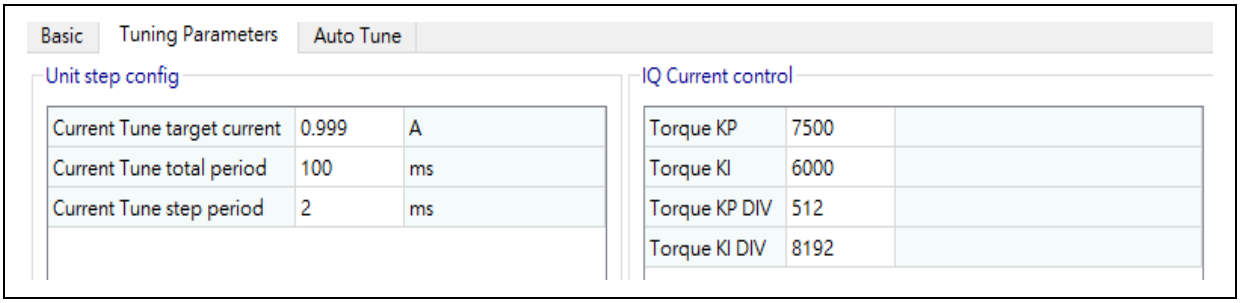
Follow the steps bellow:

STEP-1: Select “IQ tune” through the drop-down menu.

Figure 28. Select IQ tune

STEP-2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 29.

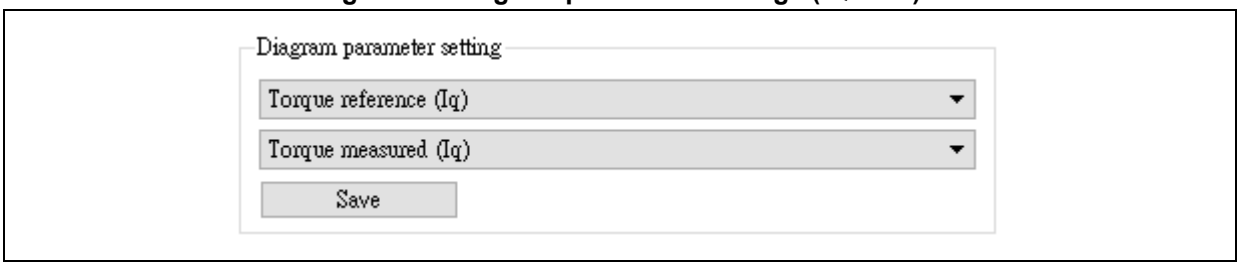
Figure 29. Q-axis current PID and step current parameters



STEP-3: Click on “Start Motor”.

STEP-4: Set “Torque reference(Iq)” and “Torque measured(Iq)” in “Diagram parameter setting”, and then click on “Save”.

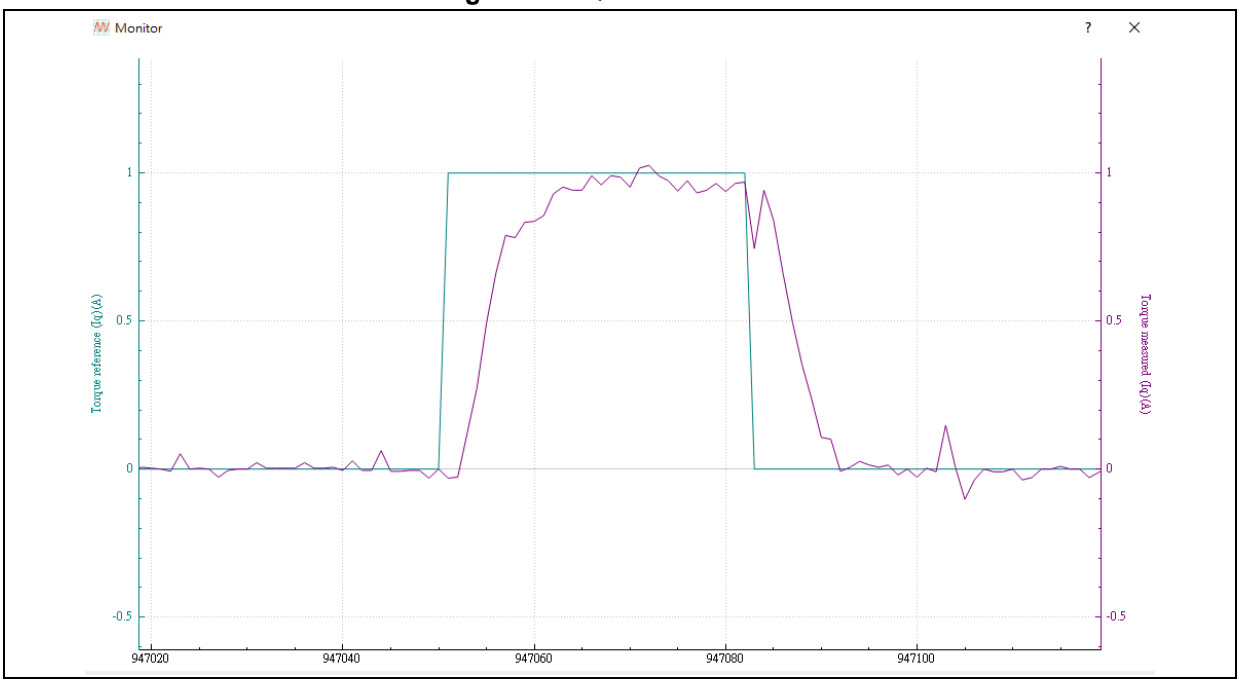
Figure 30. Diagram parameter settings (IQ tune)



STEP-5: Click on  icon to generate waveforms. For more information, please refer to *AN0063 AT32 Motor Monitor Application Note*.

STEP-6: Check whether the current response is as expected, as shown in Figure 31. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 31. IQ tune waveform



STEP-7. After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 32.

Figure 32. Q-axis current PI control parameter macro definition

motor_control_drive_param.h

```
252  #define PID_IQ_KP_DEFUALT              (7500)
253  #define PID_IQ_KI_DEFUALT              (6000)
254  #define PID_IQ_KP_GAIN_DIV             (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG         (LOG2(PID_IQ_KP_GAIN_DIV))
256  #define PID_IQ_KI_GAIN_DIV             (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG         (LOG2(PID_IQ_KI_GAIN_DIV))
```

3.9 Current loop control

In this mode, the current loop control is used to control torque by tuning the target current. The results are reflected through waveforms.
Follow the steps bellow:

STEP-1: Select “Torque Control” through the drop-down menu.

STEP-2: Set software control as control source and configure the target current reference (Torque reference).

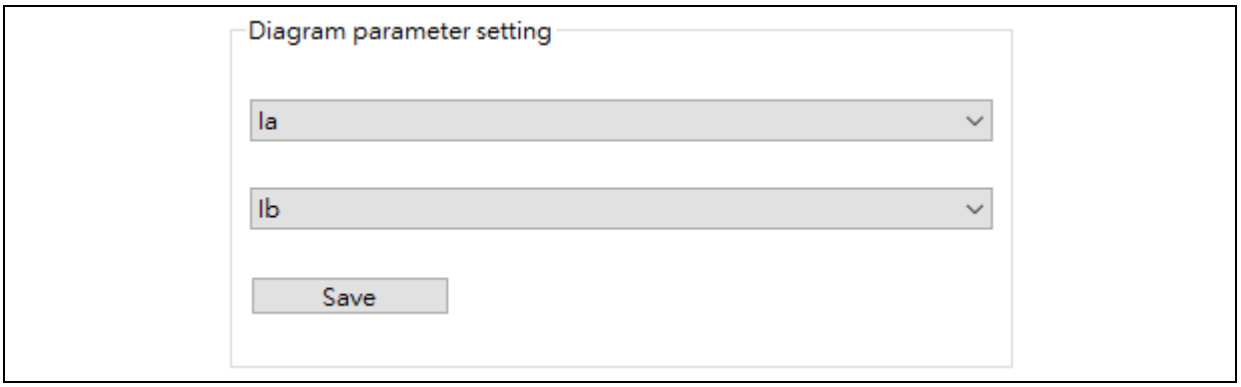
Figure 33. Set target current

Torque reference (Iq)	0.100	A
Flux reference (Id)	0.000	A

STEP-4: Click on “Start Motor”.

STEP-5: Set “Ia” and “Ib”, and then click on “Save”.

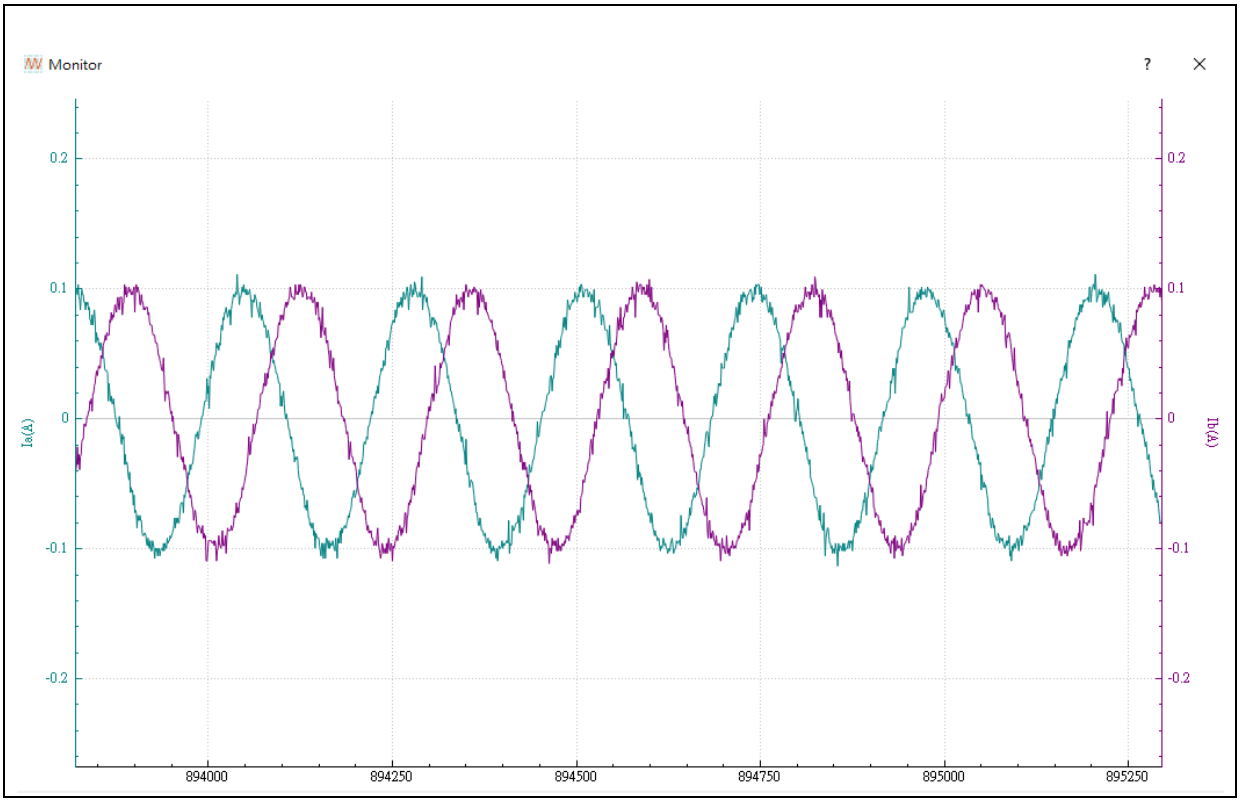
Figure 34. Parameter settings (current loop debugging)



STEP-6: Click on “ ” to open a waveform window.

STEP-7: Check if the current response is as expected, as shown in Figure 35.

Figure 35. Current loop debug waveform



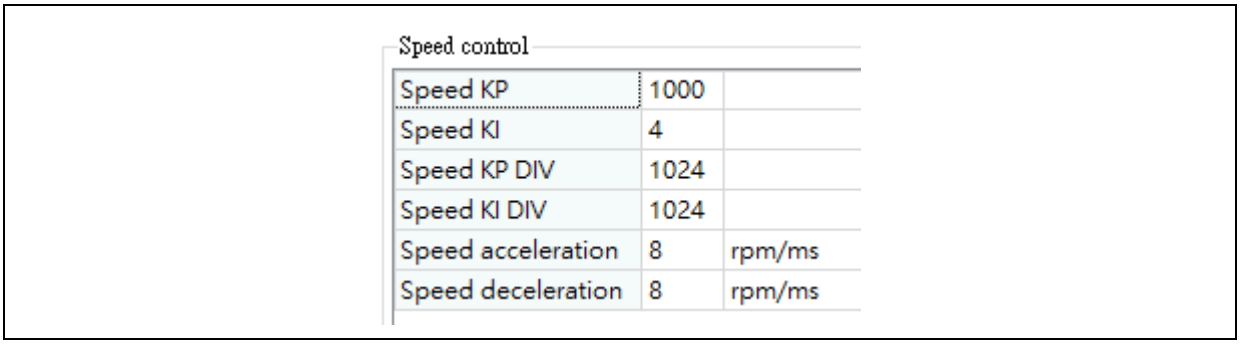
3.10 Speed loop control

In this mode, users can set the speed PID parameters, acceleration and deceleration, and check the response through waveform. Follow the steps bellow:

STEP-1" Select "Speed Control" through the drop-down menu.

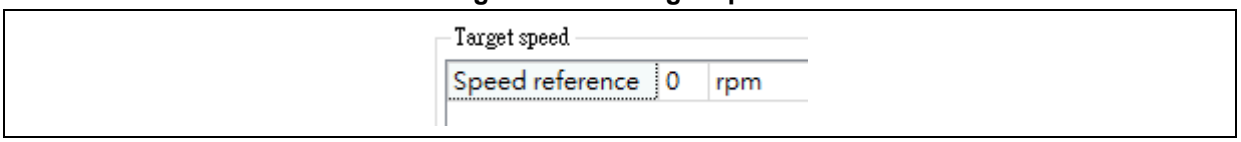
STEP-2: Go to "Tuning Parameters" window, and set speed PID parameters, acceleration and deceleration.

Figure 36. PID parameters, acceleration and deceleration



STEP-3: Set software control as control source and configure the target speed reference (Speed reference).

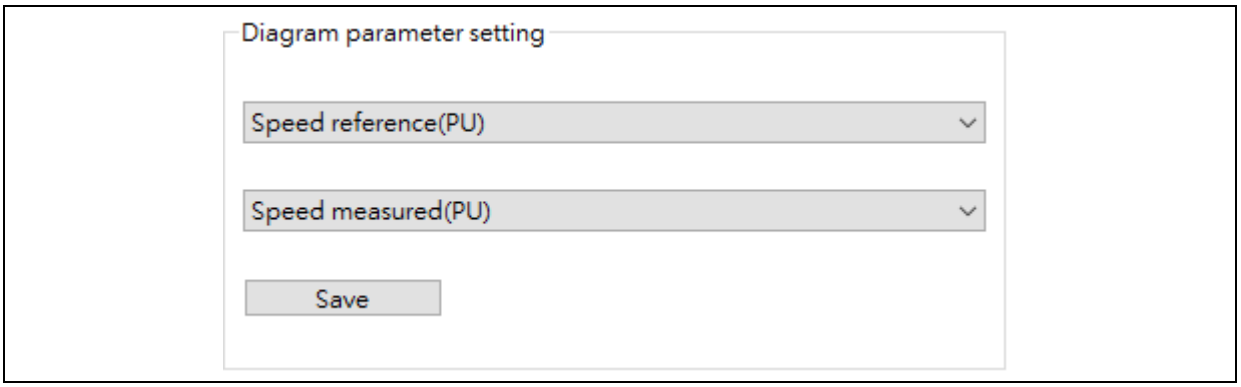
Figure 37. Set target speed



STEP-4: Click on "Start Motor".

STEP-5: Set "Speed reference" and "Speed measured".

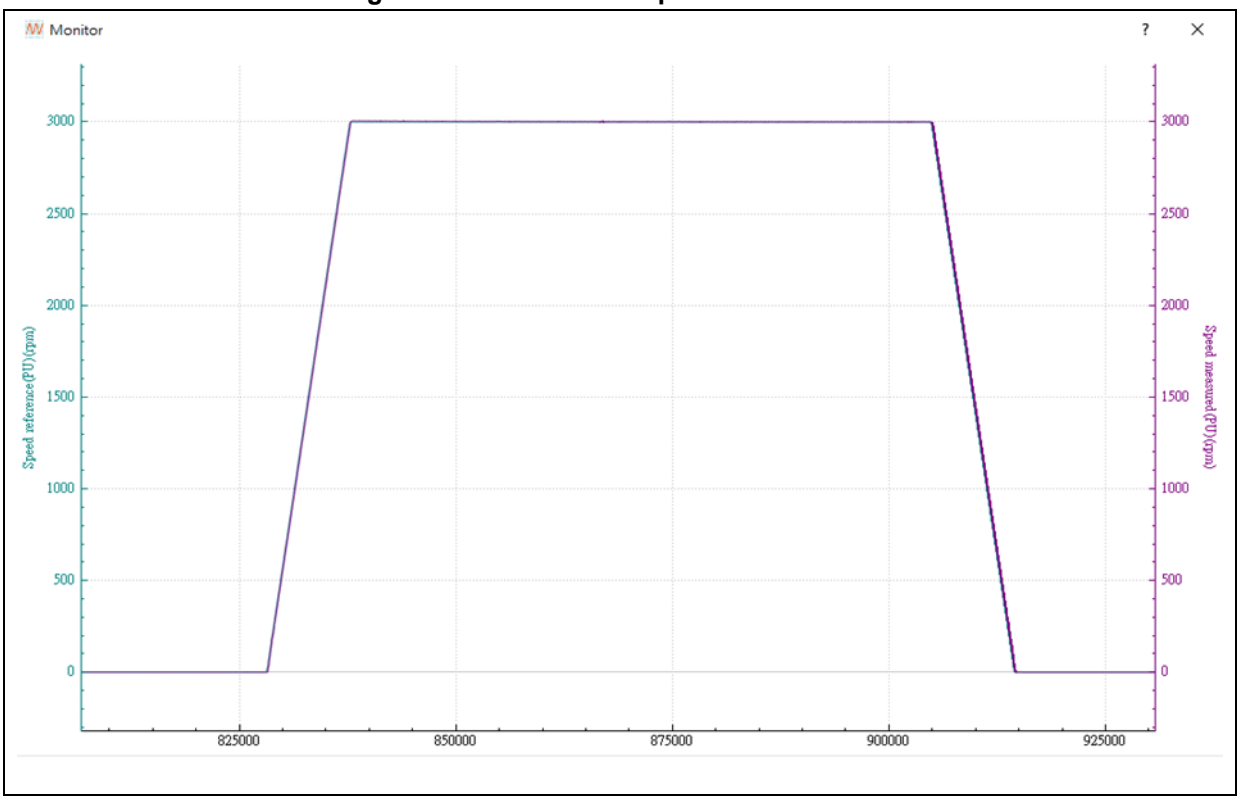
Figure 38. Diagram parameter settings (speed control)



STEP-6: Click on  to open a waveform window.

STEP-7. Check if the speed response is as expected, as shown in Figure 39. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 39. Waveform in speed control mode



Note: The maximum displayed values for speed waveforms (Speed Reference (PU) and Speed Measured (PU)) are defined as 1.25 × MAX_SPEED_RPM, with the minimum being -1.25 × MAX_SPEED_RPM. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to set MAX_SPEED_RPM to the motor's rated speed to avoid display anomalies.

3.11 Position loop control

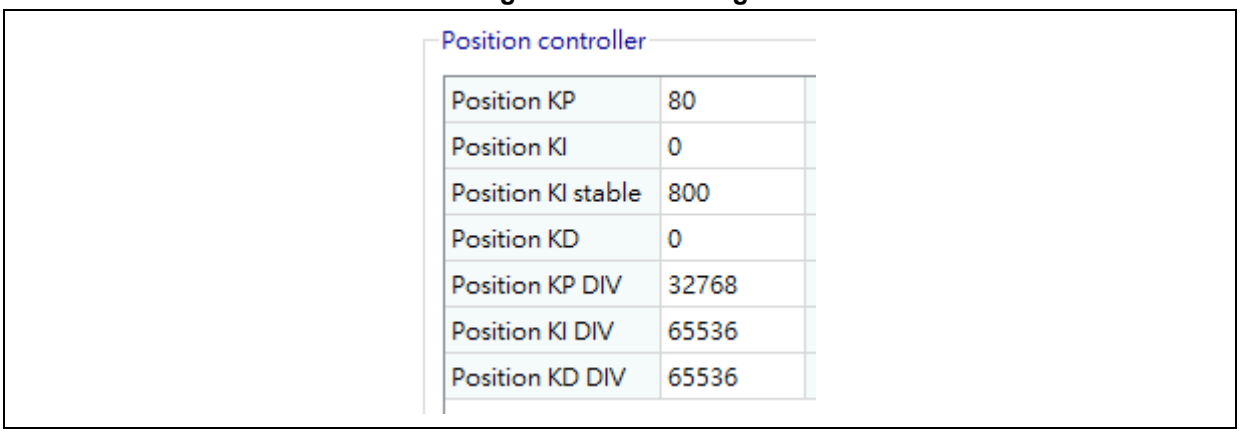
In position loop control mode, the user can set position loop PID data and view response waveforms.
Follow the steps bellow:

STEP-1: Select "Position Control" through the drop-down menu. If the motor has rotated, its angle usually will not stop exactly at the Zero degrees, so in this case, the positioning command reference and the measured position are the angle after the motor's rotation.

Figure 40. Position reference and measured position

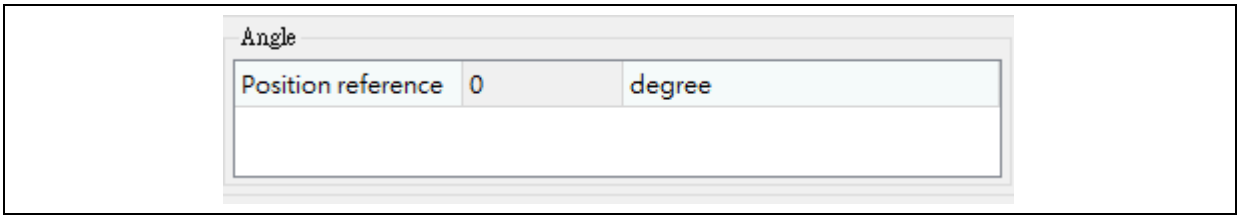
STEP-2: Set position controller PID parameters.

Figure 41. PID settings



STEP-3: Set Position reference, such as 36000 degrees.

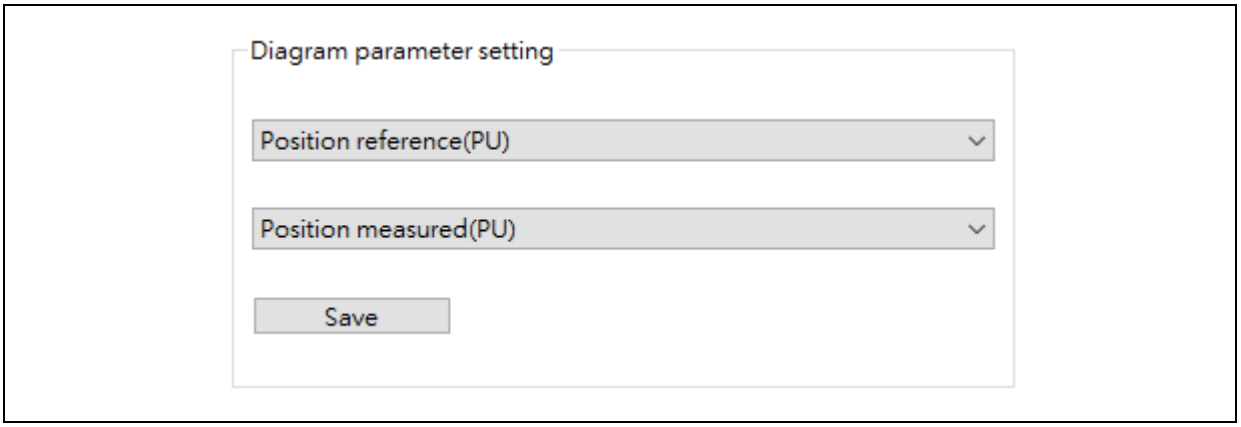
Figure 42. Set position reference



STEP-4: Click on “Start Motor”.

STEP-5: Set “Position reference(PU)” and “Position measured(PU)”, and then click on “Save”.

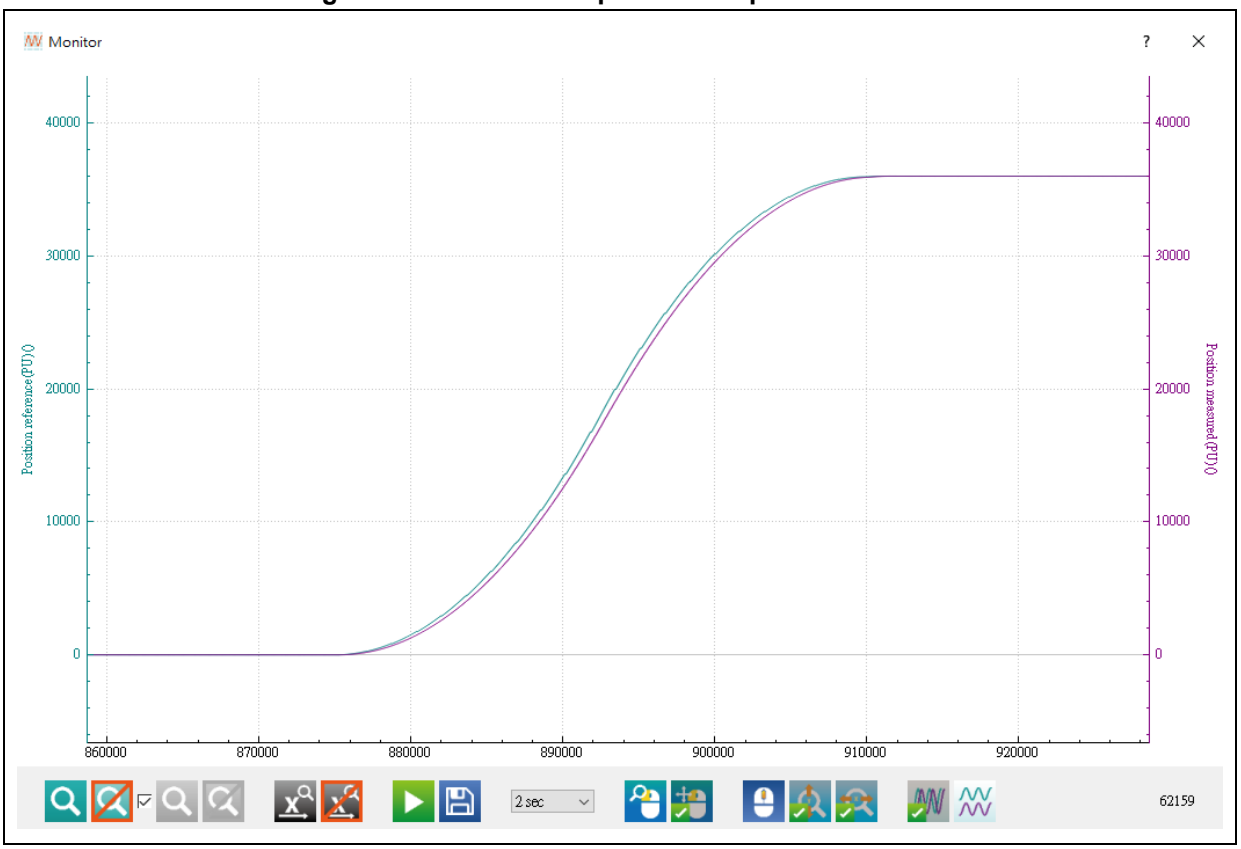
Figure 43. Diagram parameter setting (position loop)



STEP-6: Click on  to open a waveform window.

STEP-7: Check whether the position response is as expected, as shown in Figure 44. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 44. Waveform in position loop control mode



Note: The maximum displayed values for position waveforms (Position reference (PU) and Position measured (PU)) are defined as 1.25 × MAX_POSITION_ANGLE, with the minimum being -1.25 × MAX_POSITION_ANGLE. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to adjust MAX_POSITION_ANGLE value appropriately to avoid display anomalies.

3.12 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM)

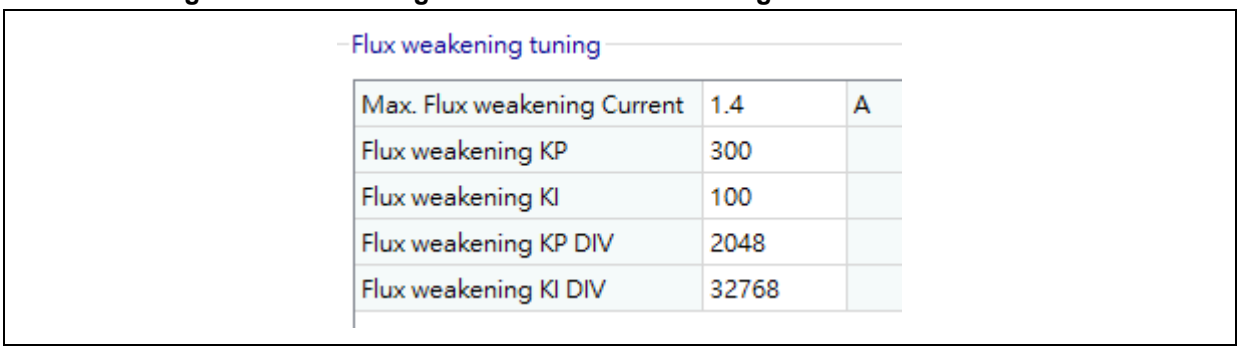
This mode supports Surface-Mounted Permanent Magnet Synchronous Motors (SPMSM). During speed-loop control, it injects negative d-axis current commands to weaken the permanent magnet flux linkage, thereby extending the motor's operational speed range. Adjustable parameters include the flux-weakening control (FWC) PID gains and the maximum negative d-axis current limit. After tuning, the dynamic response can be visualized via waveform plotting. Detailed implementation steps are as follows:

Step 1: Select "Speed Control" through the drop-down menu.

Step 2. Navigate to the Tuning Parameters page, configure the maximum d-axis (flux-weakening) current command and adjust the flux-weakening PID parameters.

Note: Configure the maximum d-axis (flux-weakening) current command according to the motor's rated specifications. Exceeding this limit may cause irreversible demagnetization of permanent magnets or thermal damage due to excessive copper/core losses.

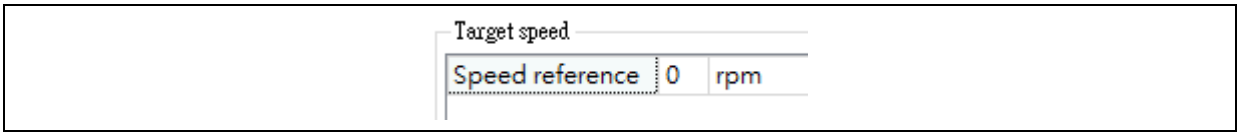
Figure 45. FWC PID gains and the maximum negative d-axis current limit



Step 3. Set Control Source to Software Control and input the Speed Reference value. For example: 8000rpm.

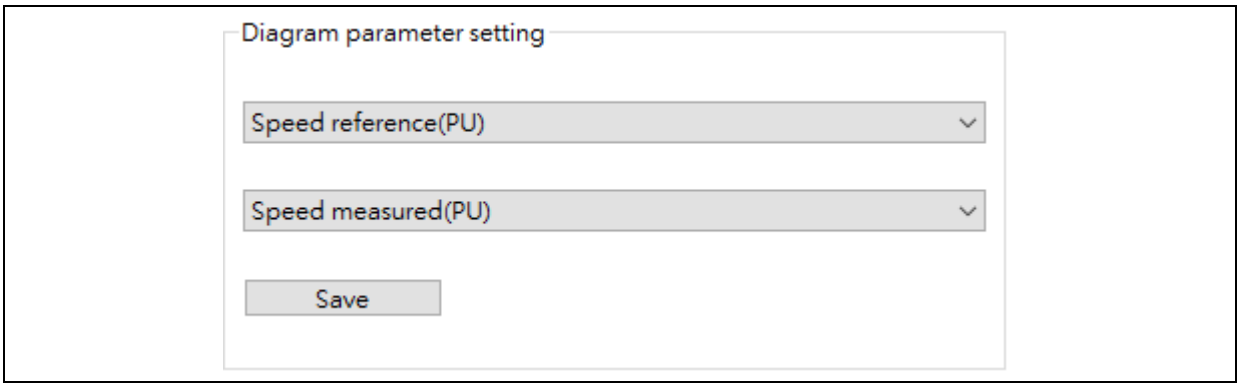
Note: The flux-weakening control (FWC) is triggered only when the motor speed exceeds the base speed (i.e., the maximum sustainable speed under rated flux linkage).

Figure 46. Target speed setting (FWC)



Step 4: Select Speed reference(PU) and Speed measured(PU), and click on “Save”.

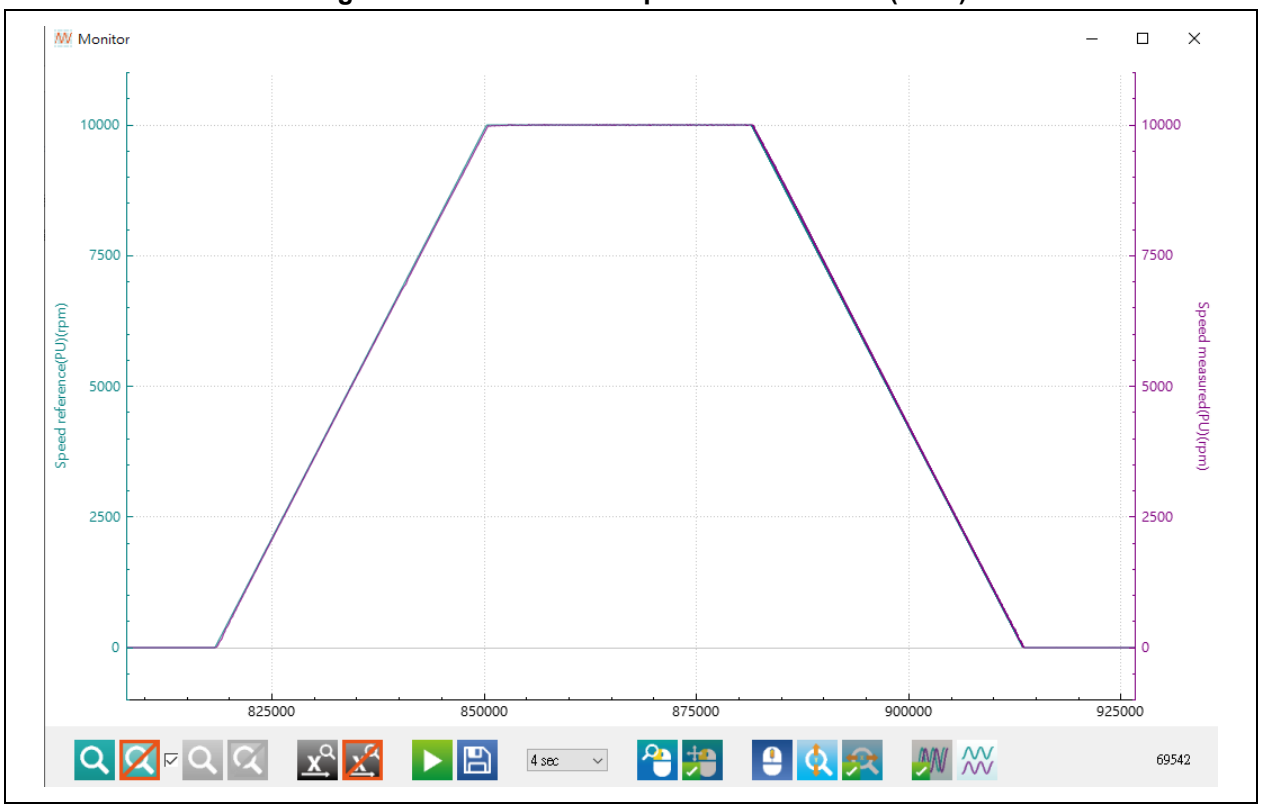
Figure 47. Diagram parameter settings (speed control)



Step 5: Click  icon to generate waveforms.

Step 6: Click on “Start motor” to check whether the speed response is as expected, as shown in Figure 48. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 48. Waveforms in speed control mode (FWC)



3.13 MTPA/MTPV Control (Interior Permanent Magnet Synchronous Motor, IPMSM)

MTPA (Maximum Torque Per Ampere) is an optimized control strategy for Interior Permanent Magnet Synchronous Motors (IPMSM) in the base speed region (constant torque zone). Its goal is to minimize the stator current amplitude for a given torque demand by adjusting the d-axis and q-axis current components (Id, Iq), ensuring the motor delivers the target torque with minimal current.

MTPV (Maximum Torque Per Voltage) is a deep flux-weakening control strategy for the high-speed region, where the motor maximizes torque output under voltage constraints by adjusting Id and Iq. At high speeds, the back-EMF increases, reducing voltage margin. This necessitates increasing the negative Id current to weaken the magnetic field and extend the speed range.

Why MTPA/MTPV is exclusive to IPMSM:

IPMSM relies on salient-pole characteristics (Ld < Lq) to generate reluctance torque, which is critical for MTPA/MTPV.

Surface PMSM (SPMSM) cannot leverage this due to structural limitations (Ld ≈ Lq) making MTPA/MTPV ineffective.

Implementation Steps for MTPA/MTPV Table Generation:

Step 1. Define parameters for table generation: Maximum torque, Maximum speed, Torque step size and Speed step size. If a specific torque/speed value is not explicitly listed in the table, the corresponding d-axis and q-axis current components (Id, Iq) will be calculated via interpolation.

Step 2. Generate Iq_table and Id_table (as shown in Figure 49):

Example parameters for table generation:
Maximum torque: 5 Nm
Torque step size: 0.5 Nm
Maximum speed: 5000 rpm
Speed step size: 500 rpm

Figure 49. Example table for Iq_table and Id_table

Torque(Nm) / Speed(rpm)	0	0.5	....	5
500	Iq_{pu1}	Iq_{pu2}	....	....
1000	....	....	....	....
⋮	....	....	....	....
5000	....	....	....	....

Torque(Nm) / Speed(rpm)	0	0.5	....	5
500	$-Id_{pu1}$	$-Id_{pu2}$	....	....
1000	....	....	....	....
⋮	....	....	....	....
5000	....	....	....	....

Note: The conversion formula between per-unit current values and actual physical current values is:

$$I(A) = current\_base \cdot {}^{I_{pu}}/_{32767}$$

Step 3. Configure Motor Type as SPMSM in motor_control_param.h, Disable FIELD_WEAKENING and IPM_MTPA_MTPV_TABLE (as shown in Figure 50)

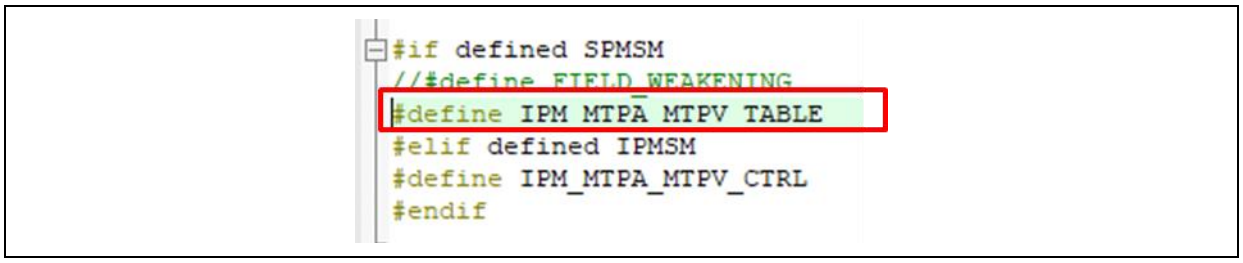
Figure 50. Macro definitions for MTPA/MTPV table generation

```
95  /* motor type */
96  #define SPMSM
97  //#define IPMSM
98
99
100 #if defined SPMSM
101 //#define FIELD_WEAKENING
102 //#define IPM_MTPA_MTPV_TABLE
103 #elif defined IPMSM
104 #define IPM_MTPA_MTPV_CTRL
105 #endif
```

Step 4. Mount the motor on the dynamometer platform and verify mechanical alignment to minimize vibration and torque measurement errors. Complete speed-loop tuning to ensure stable motor operation under both no-load and load conditions.

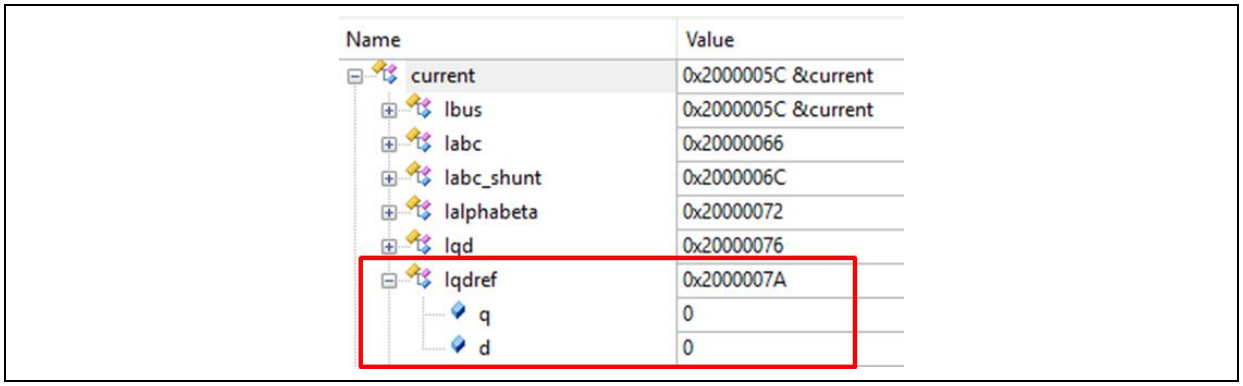
Step 5. Enable IPM_MTPA_MTPV_TABLE Macro and entering the debug mode

Figure 51. Enable the macro definition IPM_MTPA_MTPV_TABLE



Step 6. Configure -Id current reference (current.Iqdref.d) via watch window, The Iq reference (current.Iqdref.q) is automatically generated by the speed PI controller based on the speed error and tuning gains.

Figure 52. Set the per-unit value for the Id current command

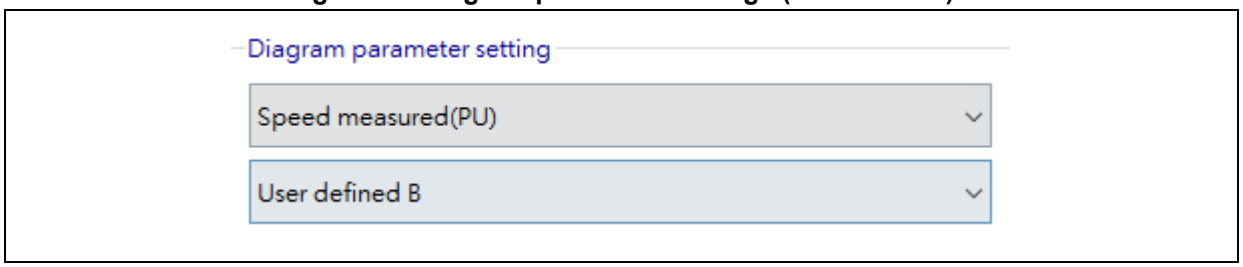


Step 7. Align the motor encoder to ensure accurate speed feedback for control stability and enter the speed closed-loop control to prepare for MTPA/MTPV table generation. (Prior to table creation, connect the motor to a dynamometer and run it at high speed for 10-20 minutes to achieve thermal stabilization)

Step 8. Follow the table in Figure 49 to identify the optimal Iq and -Id values that minimize the resultant current I_S across different speeds and torques, starting with mid-range conditions (2500 rpm, 2.5 Nm), while monitoring whether the speed reaches the commanded value and increasing the -Id command if necessary to enhance speed performance.

Step 9. Observe the speed waveform and Is waveform (User-defined B) on the UI to identify the minimum resultant current I_S that achieves the target speed/torque, which corresponds to the optimal Iq and -Id values.

Figure 53. Diagram parameter settings (MTPA/MTPV)



Step 10. After completing the table generation, populate the table indices into the macro definitions in the motor_control_param.h header file as shown in Figure 54. The values for MTPA_MTPV_TABLE_ROWS_NUM and MTPA_MTPV_TABLE_COLS_NUM can be determined from Figure 55.

Figure 54. MTPA/MTPV table indices

```
/* MPTA MTPV table parameters for IPMSM use only */
#define MTPA_MTPV_TABLE_MIN_SPEED        (500)   /* rpm */
#define MTPA_MTPV_TABLE_MAX_SPEED        (5000)   /* rpm */
#define MTPA_MTPV_TABLE_SPEED_STEP       (500)    /* rpm */
#define MTPA_MTPV_TABLE_MAX_TORQUE       (5.0f)   /* Nm */
#define MTPA_MTPV_TABLE_TORQUE_STEP      (0.5f)   /* Nm */
#define MTPA_MTPV_TABLE_ROWS_NUM         (10)     /* speed steps */
#define MTPA_MTPV_TABLE_COLS_NUM         (11)     /* Torque steps */
```

Figure 55. MTPA/MTPV table row and column count

	COLS	1	2	3	4	5	6	7	8	9	10	11
	Torque→											
ROWS	Speed ↓	0	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5
1	500											
2	1000											
3	1500											
4	2000											
5	2500											
6	3000											
7	3500											
8	4000											
9	4500											
10	5000											

Step 11. After table creation, populate the Iq_table and Id_table values into the Id_MTPA_LUT[] and Iq_MTPA_LUT[] matrices in the MTPA_MTPV_table.c file.

Step 12. In the motor_control_param.h header file, select the IPMSM macro and define IPM_MTPA_MTPV_CTRL to enable torque control mode. Inputting torque commands will trigger table-based MTPA/MTPV control.

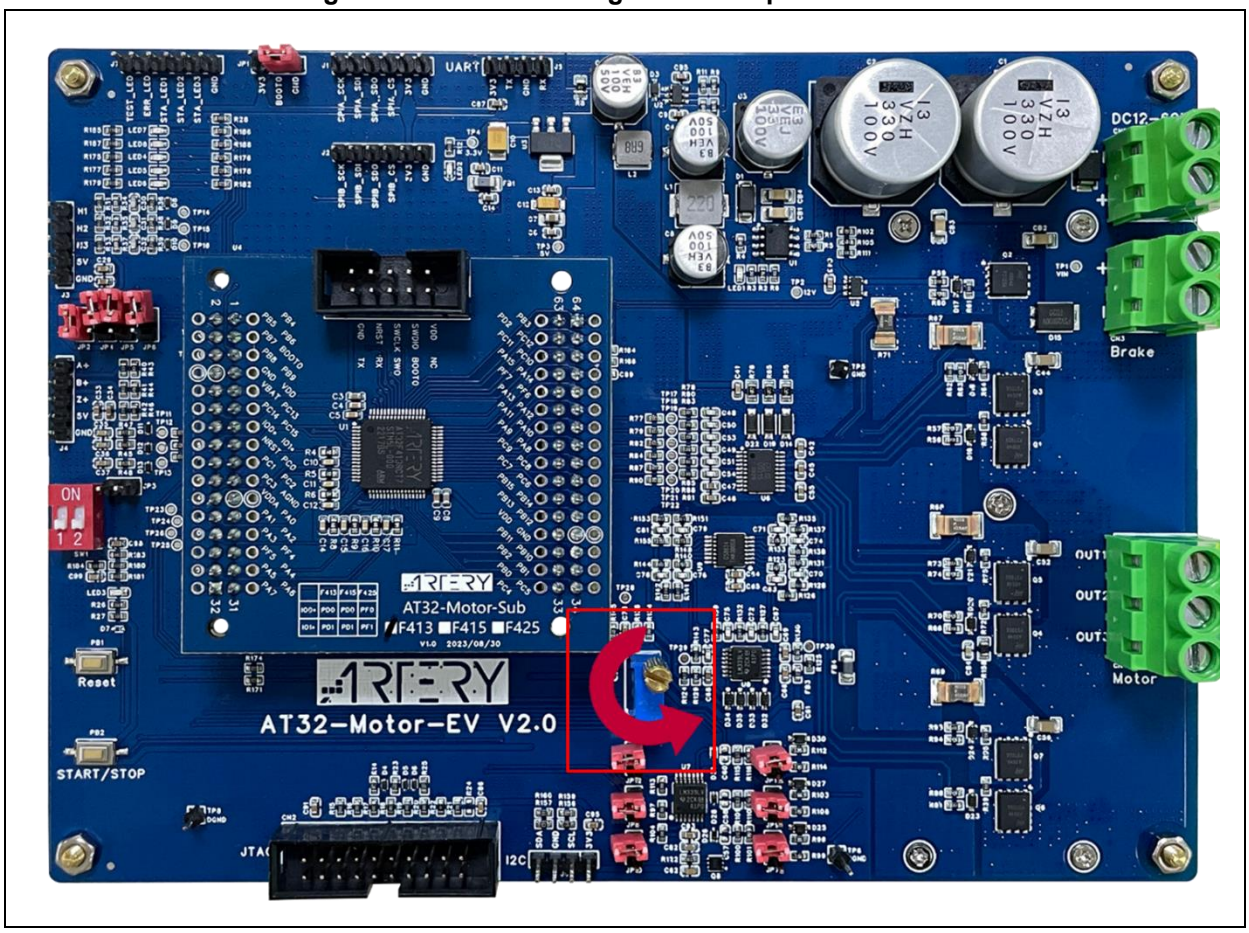
Step 13. During torque control, verify the Torque reference waveform and the magnitudes of the Iq/Id current waveforms using a UI monitoring tool to ensure they align with expectations.

Step 14. When using IPM_MTPA_MTPV_CTRL, the speed loop output command is Torque reference (Nm), whereas in SPMSM control it is Iq reference. Recalibrate the speed PI control parameters for this mode due to the fundamental difference in command logic.

3.14 External voltage control/software command control

This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 56. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 57) or through PC software (see Figure 58). Control parameters are different for different control modes, such as the target speed (as shown in Figure 58) in speed control mode, or torque reference (as shown in Figure 59) in torque control mode.

Figure 57. Control source definition (software control (upper)/external voltage control (lower))

```
motor_control_drive_param.h
183
184 /********************************* Control-related parameter ************************
185 #define PWM_FREQ                  (30000)                          /*!< Hz */
186 #define MOTOR_CONTROL_MODE        (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE               (ctrl_source_type)(CTRL_SOURCE_SOFTWARE)
188 #define UI_UART_BAUDRATE          (1500000UL)
189 #define UI_SAMPLE_CYCLE           (1)

motor_control_drive_param.h
183
184 /********************************* Control-related parameter ************************
185 #define PWM_FREQ                  (30000)                          /*!< Hz */
186 #define MOTOR_CONTROL_MODE        (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE               (ctrl_source_type)(CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE          (1500000UL)
189 #define UI_SAMPLE_CYCLE           (1)
```

1) Select “External voltage control” as a control source

Select “external voltage control” in “Control source”. In this mode, it is possible to adjust speed or torque value by setting external voltage values. The “Target speed” and “Torque reference” area will generate the control speed or torque calculated based on the current voltage.

Note: The “Target speed” and “Torque reference” cannot be modified in “external voltage control” mode.

Figure 58. Speed control mode (external voltage control)

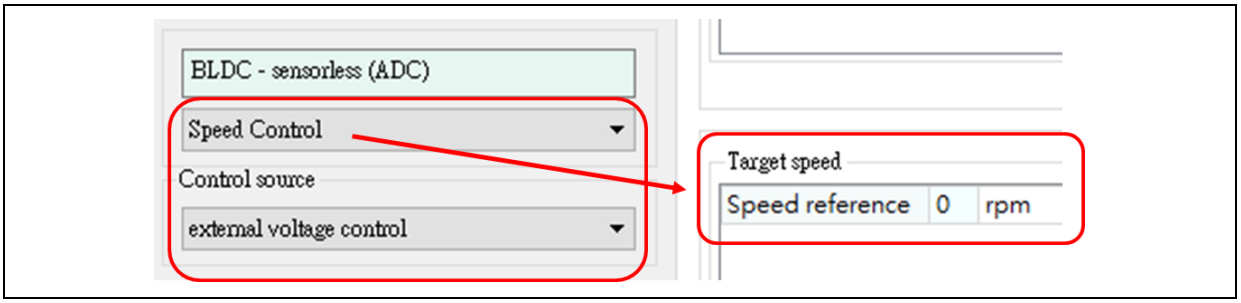
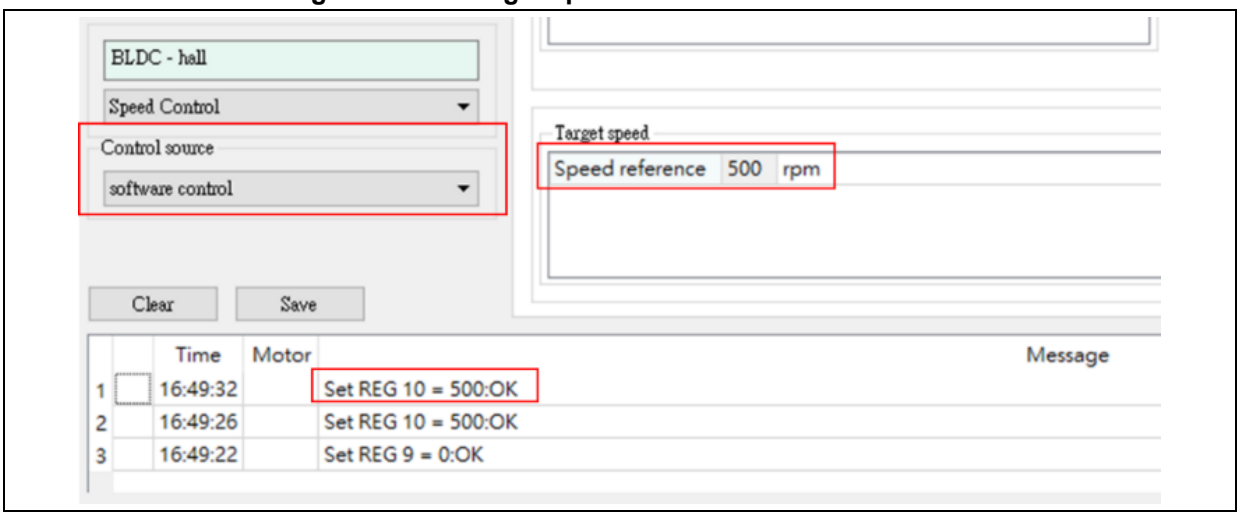


Figure 59. Torque control mode (external voltage control)

2) Select “Software control” as a control source

The software control mode is selected by default, as shown in Figure 60. In this mode, the user can adjust the motor speed and torque by changing the target speed/torque reference. The “message” area will show whether the changing operation is successful or not.

Figure 60. Set target speed in software control mode



4 Revision history

Table 25. Document revision history

Date	Version	Revision note
2024.05.24	2.1.0	Initial release.
2024.12.27	2.1.1	1. Added AT32F435/437 series and AT32L021 series support; 2. Added description of IAR environment and *icf file modification for Flash configuration; 3. Updated definitions, code and some descriptions.
2025.04.11	2.1.2	Added support for the AT32F425 series; Added new functionalities including field-weakening control, and MTPA/MTPV table generation and lookup features
2025.10.09	2.1.3	Added new supported models, fixed flash configuration table and parameter updates

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.